

Chapter 1

Introduction

CHAPTER 1. INTRODUCTION

1.1 Background –

Ransomware is classified as any malware which forces users to pay a ransom payment in exchange for regained access to their system (Humayun et al., 2020). This malicious software has caused significant disruptions across industries such as finance, energy, defence, and even personal computing. Over the past decade, ransomware growth has escalated at alarming rates, with evolving variants adapting to the rapidly changing cyber landscape (Karim, 2024). A Europol report (Yuryna Connolly et al., 2020) flagged ransomware as a "dominant threat" affecting both private and public organisations globally. Despite efforts to mitigate these attacks—such as the deployment of anti-virus software and policy updates—ransomware incidents continue to grow, leading to severe financial losses, operational downtime, and reputational damage.

Existing detection methods, including signature-based and anomaly-based techniques, have struggled to keep up with the dynamic nature of ransomware. Signature-based detection, for instance, relies on identifying known patterns of attack, rendering it ineffective against recently discovered variations of zero-day ransomware variants (LaRocque et al., 2024). Similarly, traditional anomaly-based approaches have been limited by high false-positive rates and scalability challenges. These shortcomings highlight the urgent need for more advanced, adaptive, and intelligent detection mechanisms.

This project aims to address these limitations by leveraging machine learning, a cutting-edge approach capable of analysing large datasets and identifying subtle patterns associated with ransomware behaviour. Unlike traditional methods, machine learning models can adapt to evolving threats, making them well-suited for detecting previously unseen ransomware strains. By evaluating and implementing such techniques, this project seeks to contribute to the development of more robust and proactive ransomware detection systems.

1.2 Problem statement

Traditional malware detection tools in the form of signature-based and static analysis are no longer effective at detecting today's ransomware attacks—especially zero-day types that do not have any resemblance to a familiar pattern. These traditional approaches are unable to match the growing complexity of ransomware behaviour, thus resulting in late detection, false positives, and poor response times.

This project closes the gap in intelligent, adaptive detection technology by examining how to use machine learning models to identify ransomware as opposed to other malware in real time. The main challenge is to develop a system that will generalize from the data that is known, but correctly identify unknown or mutated ransomware. This issue must be solved to enhance the cybersecurity stance of organizations against evolving threats at an increasingly rapid pace.

1.3 Aims and objectives

Aims:

- To develop a machine learning-based intelligent detection system that is capable of detecting the presence of ransomware, categorising it into its specific family strain, and identifying the threat level of the severity.
- Design that supports proactive cybersecurity efforts by delivering real-time information and alert systems through an interactive web interface.

Objectives:

1) Data Collection and Preparation

- Obtain publicly available labelled samples of ransomware (with family labels), other malware, and benign files. Clean, normalize, and balance to obtain quality input for training.

2) Feature Engineering

- Mine static and behavioural characteristics (e.g., API calls, file I/O, execution traces) that are relevant for ransomware family identification and attack severity estimation.

3) Model Development

Design and develop machine learning models that:

- Identify the presence of ransomware in input samples.
- Accurately categorise ransomware into known family strains.
- Estimate the degree of severity or effect of the threat found based on behavioural indicators.
-

4) Model Optimisation

- Use bio-inspired metaheuristic methods such as genetic algorithms in an effort to optimise model hyperparameters and improve detection precision, family classification accuracy, and severity scoring.

5) Model Evaluation

- Assess model performance with typical classification metrics (accuracy, precision, recall, F1-score) and confusion matrices, paying close attention to the model's capacity for generalising to new, unseen samples.

6) Web-Based Interface

- Develop a Flask-powered user interface that delivers real-time notifications, shows classification results, and digests severity information in an intuitive, easy-to-understand manner.

7) Documentation and Reporting

- Maintain accurate technical records and write a comprehensive project report encompassing methodology, implementation decisions, experimental results, issues, and future recommendations.

1.4 Solution Strategy

To implement the project goal of detecting ransomware, determining its type, and evaluating its impact, a modular machine learning framework is under development. The system incorporates supervised learning processes, feature extraction mechanisms, and optimization methods into a systematic developmental structure. The project is segmented into repeatable cycles that correlate with Agile methodology, hence the provision of adaptability and ongoing improvement.

1.4.1 Data-Driven Detection Pipeline

The fundamental component of the solution is:

- Gathering of data from publicly available malware datasets (e.g., MalMem2022, VirusTotal, MalwareBaazaar).
- Feature engineering covers both static and dynamic features, including API calls, levels of entropy, and system actions.
- Model training through supervised classifiers such as Random Forests, SVM, or deep learning models to detect ransomware presence and classify families.
-

1.4.2 Hyperparameter Optimisation and Severity Analysis

For higher accuracy, bio-inspired metaheuristic algorithms such as genetic algorithms will be employed to optimise model hyperparameters. Another severity estimation module will analyse feature impact (e.g., process tampering, encryption activity) to label threats.

1.4.3 User Interface and Real-Time Feedback

A web interface using Flask will be created to present real-time detection output, ransomware family identification, and severity levels. This is for ease of access, readability, and user interaction.

1.4.4 Iterative and Agile Development Methodologies.

An Agile methodology is followed throughout the project lifecycle. The work is divided into sprints, and each sprint focuses on key deliverables (e.g., data preprocessing, model development, UI integration). Continuous evaluation allows model and system feature enhancement.

CHAPTER 1. INTRODUCTION

1.5 Summary of contributions and achievements

This project delivers a ransomware detection tool based on machine learning, which has the capability to detect the presence of ransomware, classify detected samples into corresponding family strains, and give estimates on threat severity. The tool exploits behaviour-based and static feature analysis using supervised learning models trained over real datasets.

A significant addition is the inclusion of a bio-inspired optimisation technique (genetic algorithm) for optimising model hyperparameters to fine-tune classification accuracy and threat level prediction. Real-time visualisation of detection output and threat severity scores was provided using a basic web interface developed using Flask.

Initial tests have shown the system to easily identify ransomware from a number of other forms of malware, with robust performance statistics in detection and classification processes. The findings confirm proactive threat management practices and indicate the prowess of adaptive machine learning algorithms in dynamic cyber threat environments.

1.6 Organization of the report

This report is structured around seven main chapters, which deal with the main themes of the project.

Chapter 1 – Introduction: Presents the background, gives the research problem, includes the objectives and aims of the project, and gives an overview of the solution strategy. It also includes the project contributions and describes the report organisation.

Chapter 2 – Literature Review: Summarises existing work and methodologies toward ransomware detection, both classic and machine learning-based. Addresses gaps in comprehensive solutions and necessitates the choice of this project.

Chapter 3 – Methodology: Describes the ransomware detection system design, data collection, feature engineering, model construction, optimisation techniques, and rationale for chosen methods.

Chapter 4 – Results: Reports and analyses the performance of the implemented system, briefly overviewing evaluation metrics as well as model performance in detecting ransomware and ransomware family classification.

Chapter 5 – Discussion and Analysis: Explains the results within a wider context, connecting the implications, the limitations, and the importance of the findings.

Chapter 6 – Conclusions and Future Work: Recaps the project's success and suggests potential future improvements, e.g., from other data sources or investigating other detection methods.

Chapter 7 – Reflection: Provides a personal reflection on the learning experience, development process, and skills gained during the project. Appendices: Include supplementary information like sample code, system diagrams, and other assessment results.

Chapter 2

2.1 Literature Review

2.1.1 Introduction

Ransomware has earned its title as one of the most destructive cybersecurity threats by targeting individuals, businesses, and governments worldwide (Brook, 2024). Ransomware is a rapidly evolving malware type, with each family variant instance divergent from its predecessor. Consequently, traditional detection mechanisms like signature-based detection are becoming redundant, especially towards unknown ransomware (Arief et al., 2020). Furthermore, modern ransomware employs advanced obfuscation techniques, such as encryption, which allow it to hide inside systems and continue to have a damaging effect.

This chapter will examine the significant research conducted by scholars surrounding the complexities of ransomware and the effectiveness of existing detection techniques. It will also provide a comprehensive review of the challenges in ransomware detection by understanding the methods various ransomware families use to avoid detection. Subsequently, the literature review aims to provide clarification into the utilisation of machine learning techniques and assess whether implementation will contribute to the alleviation of industry and domestic ransomware fatalities.

2.1.2 Overview of ransomware families

Categorisation of ransomware is consistent with two major types: locker ransomware (system lockers) and cryptographic ransomware (cryptoware). The former locks the user's device generally by restricting input functions like the screen or keyboard. The latter incorporates cryptography techniques, targeting important files or documents on the system that cannot be released without the correct key (Gomez-hernandez et al., 2017).

Several cryptoware families have risen over the years, with some garnering significant attention due to the scale of their attacks. WannaCry, a famous cryptographic ransomware strain, described as “taking the world by storm”, exploited the EternalBlue vulnerability in Windows systems, which affected 200,000 computers across 150 universities. The magnitude of the attack crippled hospitals, businesses, and even government institutions (Mohurle & Patil, 2017). Similarly, TeslaCrypt primarily targeted gaming files before its operators suddenly shut it down in 2016 (Kharraz et al., 2016). Cerber introduced a Ransomware-as-a-Service model (RaaS), which enabled cybercriminals to distribute ransomware easily with minimal technical knowledge. Meanwhile, Locky and CryptoWall spread rapidly through phishing campaigns and email attachments that had malicious code embedded in them (Hunt, 2016).

Even though these ransomware families received a lot of attention, newer strains of the virus are still appearing and continue to target particular industries and regions. Maze ransomware, for example, threatened victims with public data releases until ransom payments were made by combining encryption and data exfiltration (Gandotra et al., 2021), which threatens to leak sensitive data (Ullah et al., 2017). Similar to GandCrab, REvil (Sodinokibi) was in charge of well-known breaches, such as assaults on managed service providers and supply chains (Balamurugan et al., 2024).

(Huang & Zhang, 2024) utilised the Jaccard index and cosine similarity to identify significant similarity between the characteristics used by various ransomware groups. The findings concluded that less mainstream ransomware families incorporate similar characteristics as more prominent ones, highlighting attackers’ reuse/enhancement of existing code that can circumvent modern cybersecurity safeguards. *Table 1.1* displays some of the ransomware described in this literature, their architecture, and utilised algorithms.

Family	Method of infection	Encryption algo	Feature	Familiarity
WannaCry	SMBv1 exploit (EternalBlue)	AES + RSA	Worm-like propagation	High
TeslaCrypt	Phishing, exploit kits	XOR (early), AES-256	Initially targeted gaming files	Medium

Cerber	Malicious email attachments, exploit kits	AES-256 + RSA	Ransomware-as-a-Service (RaaS), offline encryption	Medium
Locky	Phishing, macro-based emails	RSA + AES	Spread via Necurs botnet	High
Cryptowall	Phishing, exploit kits	RSA-2048	Used Tor for ransom payments	High
Maze	Phishing emails, exploit kits	RSA + ChaCha20	Combined encryption with data exfiltration, threatened public leaks	High
GandCrab	Exploit kits, malvertising	Salsa20 + RSA	Ransomware-as-a-Service (RaaS), frequently updated variants	Medium
REvil (Sodinokibi)	Exploit kits, supply chain attacks	AES + Salsa20	Targeted managed service providers and supply chains	High

Table 1.1

The inability of conventional signature-based detection methods to identify zero-day ransomware is highlighted by the growing complexity of ransomware families. Therefore, in order to detect malicious activity before encryption starts, contemporary detection systems rely on machine learning, behavioural analysis, and heuristic-based anomaly detection. Although current research correctly identifies some of the unfamiliar variants, the use of public datasets for detection training might not fully encompass all ransomware. Building on previous research, future studies could incorporate a greater range of data sources, including collaborations with cybersecurity companies and proprietary information. This would improve detection models and response tactics by offering a more comprehensive picture of ransomware threats.

2.1.3 Existing ransomware detection techniques

In order to protect users and stop the infection from spreading, detection is crucial before a ransomware assault takes place. Newly developed ransomware variants are frequently difficult for traditional signature-based detection techniques to detect, particularly when they use polymorphic and metamorphic strategies to avoid detection (Baker, 2022). In order to increase the precision and effectiveness of ransomware identification, researchers have investigated a number of detection techniques, such as heuristic analysis, behaviour-based detection, and machine learning models.

2.1.4 Signature-based detection

One of the oldest and most used techniques in cybersecurity is signature-based detection, which is typically utilised by intrusion detection systems (IDS) and antivirus software. It functions by matching a file's attributes, like the hash value, opcode sequences, or static features, with a pre-compiled database of known malware signatures (SentinelOne, 2021).

An open-source network intrusion detection and prevention system called Snort is one popular IDS that uses signature-based methods (Rubens, 2021). Incoming network data is compared to a predetermined collection of signatures that describe known malicious activity in order for Snort to function. The efficacy of Snort in identifying malware in PDF files was investigated in a study by (Khatoun et al., 2024). The study discovered that although Snort was able to identify well-known malware, it had trouble identifying polymorphic strains that utilised traffic manipulation to prevent them from matching any IDS signatures.

(Labiche & Massicotte, 2012) analysed the detection feedback in regards to network intrusion systems and discovered signature-based techniques' poor identification of correct alarms. The magnitude of alarms generated inundated the network, causing the noise to be overwhelming, and thus, "false alarms" were observed. This highlighted the struggle for IDSs to divide normal and attack traffic accurately. Furthermore, it gave way to "squealing attacks," a method adopted whereby the perpetrator creates fictitious assaults to overwhelm the network and increase the chance of it incorrectly identifying the attack/falsely accusing a normal file.

Signature-based detection's most exploitable feature is its incapability, or lack thereof, to successfully identify zero-day ransomware. By taking advantage of flaws that have not been made public, attackers

thrive in systems that rely on this form of detection by remaining unrecognised and thus dealing increasing damage (Guo, 2022).

A variety of organizations such as Adobe Systems, Rackspace, Juniper Networks, Google, and others were hit by the Aurora attacks. The attacks took advantage of several zero-day vulnerabilities that enabled attackers to compromise systems and pilfer intellectual property (Gelinas, 2010). In the same vein, the US Veterans of Foreign Wars website was compromised to initiate Operation SnowMan, a cyber espionage attack aimed at American military personnel. The perpetrators of this operation were also involved in major cyber incidents, such as DeputyDog and Ephemeral Hydra (Nath et al., 2019 as cited in Kindlund, 2014).

2.1.5 Heuristic-based detection

Cybersecurity professionals have resorted to heuristic-based detection, a more sophisticated technique that examines the traits and behaviour of possibly malicious software rather than depending just on predefined signatures, in order to overcome the drawbacks of signature-based detection (Yaseen, 2020). Heuristic detection, in contrast to signature-based techniques, can detect irregularities in file access patterns, encryption behaviour, and system alterations to uncover unidentified ransomware strains (Eisa et al., 2025). The efficacy of heuristic-based detection, its benefits over conventional techniques, and the difficulties it encounters in detecting and reducing ransomware threats will all be covered in this part.

(Nguyen et al., 2018) describes heuristic malware detection algorithms as learning how an executable file behaves by using data mining and machine learning techniques. In a study performed by Mehdi et al. (2013), the most utilised features for heuristic detection were Application Programming Interface (API) calls, OpCode, and N-grams. API calls can represent a program's behaviour by examining system call sequences to identify significant deviations and utilising anomaly detection when comparing the API call sequence to the normal behaviours of the system (Dhurjati et al., 2001). OpCode detects obfuscated, metamorphic, and unknown malware and additionally reduces the false positive rate.

Despite the utilization of the above methodologies in an effort to simplify the discovery of various categories of malware, severe issues have been raised. For instance, Santos et al. proposed a new

feature extraction method grounded on OpCode sequences that necessitates a large quantity of instances per category that machine learning-based classifiers are attempting to discriminate (malware and benign). It was noted that this amount of labelled data under real-world conditions is a challenge.

Also, heuristic systems based on the tracking of Windows API execution patterns typically have two inherent drawbacks, according to Ye et al. (2007). First, the classifier can be overwhelmed by the large number of generated rules that are needed to cover a wide range of malware behaviours, thus making the model training process even more complex. Second, it is still a great challenge to derive useful principles that can efficiently handle malware classes that have never been seen before, especially for zero-day vulnerabilities. Because of these drawbacks, heuristic-based methods that are based purely on API-level analysis have less flexibility and scalability.

In the Android environment, heuristic-based detection techniques have shown considerable promise by analysing both static and dynamic features of applications. According to Mohsen et al. (2024), static analysis methods—such as inspecting source code and configuration files—offer fast detection without the need to execute the application. However, they are often ineffective when the code is obfuscated, which limits their reliability. Dynamic analysis, however, where the app is executed in a controlled environment and its behaviour monitored, provides more robust detection, especially against runtime attacks and obfuscation (AHN et al., 2018).

In spite of this benefit, dynamic analysis is computationally and time intensive and therefore less appropriate for real-time detection uses. Mohsen et al.'s work concludes that although heuristic-based detection yields significant findings within the field of Android malware detection, its drawbacks point towards the necessity of hybrid or adaptive approaches that can offer both precision and effectiveness.

By recognising unusual encryption behaviour, unauthorised file alterations, and questionable system calls that are frequently connected to ransomware assaults, these techniques adopted by the heuristic approach enable early detection of zero-day and polymorphic ransomware (Kaspersky, 2018). However, the limitations are very apparent, as the growing complexity of malware behaviour makes it increasingly difficult to encode all notable patterns using handcrafted logic only (Ucci et al., 2018). The more dynamic the threat environment gets, the more room there is for more data-oriented and adaptive methods. Machine learning offers this potential by enabling systems to automatically learn

behavioural patterns from large data quantities, generalise across ransomware families, and continuously adapt to new attack techniques (Sgandurra et al., 2016). The following section will discuss how these learning-based models are being utilised to strengthen the detection, classification, and countermeasures against ransomware attacks.

2.1.6 Machine learning-based detection

Since the limitation of heuristic and signature-based detection approaches becomes increasingly apparent when confronted with the evolving sophistication of ransomware attacks, researchers have looked to machine learning (ML) as a smarter and more adaptive alternative (Anderson et al., 2011). In comparison to static rule-based solutions, ML algorithms possess behaviour-based detection techniques to learn automatically from past data, reveal hidden behavioural patterns of malware, and generalise to different types of ransomwares (Ikuse et al., 2016). This characteristic renders machine learning especially helpful in identifying zero-day exploits and polymorphic ransomware that commonly slip through conventional detection techniques.

An increasing amount of research work has demonstrated the promise of machine learning in this field. Sgandurra et al. (2016), for instance, have suggested EldeRan, a system that employs dynamic behavioural analysis in combination with machine learning classifiers to identify ransomware prior to the start of encryption. This solution attained an impressive rate of 96.3% in detection by targeting API calls and behaviour sequences instead of depending on static code signatures.

Likewise, Verma and Ranga (2019) applied a random forest classifier to a dataset of Android ransomware and benign apps. Their classifier's accuracy rate exceeded 96%, illustrating the excellent enhancement in classification performance that is possible with feature engineering using permissions, API calls, and intent filters. However, the research was confined to Android malware, so its applicability may be restricted to Windows-based ransomware or cross-platform malware.

In a newer study, Lopez-Martin et al. (2017) explored the use of deep learning techniques, specifically convolutional neural networks (CNNs), in malware detection. The results indicate that CNNs have the potential to automatically discover complex features from raw binary data, eliminating manual preprocessing. This automation allows greater scalability but compromises model interpretability—a

commonly referenced drawback in the application of deep learning to cybersecurity, where an understanding of the reason for alerts is necessary.

2.1.7 Bio-heuristic algorithms

The success of ML and DL models is often constrained by the quality of features and the tuning of numerous parameters. To address this, bio-inspired heuristic algorithms have emerged as effective tools for automating and optimising these tasks, enhancing both model efficiency and accuracy (Narayan & Pandiyan, 2023).

Ab Razak et al. (2018) in their research, investigated the use of bio-inspired algorithms—particularly Particle Swarm Optimisation (PSO)—for feature selection in Android malware detection. The authors identify one major problem in machine learning for cybersecurity: the existence of redundant and irrelevant features, which can worsen classification (RRRR). Their approach was to minimise the feature set of 378 permissions to 11 based on PSO and show that an optimised, reduced feature set could be used to improve model efficiency without reducing accuracy. In combination with machine learning classifiers such as AdaBoost, their method attained a 95.6% true positive rate, better than evolutionary computation and standard information gain approaches.

Similarly, Firdaus et al. (2018) explored the use of bio-inspired ANN classifiers, namely Multilayer Perceptron (MLP), Voted Perceptron (VP), and Radial Basis Function Network (RBFN), for Android malware detection, unearthing the promise of combining static analysis with bioheuristic learning techniques. The approach used a unique range-based feature selection mechanism for discovering the most important features within a total of three categories, namely permissions, directory paths, and telephony APIs. Reduced from over 500 features down to just 12 features, without a loss of accuracy in classifications, it simultaneously reduced model complexity and processing costs. Of all the classifiers compared, MLP performed best among all simulation experiments with an accuracy rate of 90% that is later validated by a real-world predictive model (Bio Analyser) with an accuracy rate of 97% based on Malgenome dataset.

However, the pure statically based approach, followed by the research, places limits on its ability for detection that starts modifying dynamically post-installation, a limitation which could be removed using integrated techniques that use dynamic or online learning along with static analysis. These papers strongly recommend and demonstrate the use of bio-inspired learning frameworks for

detection of malware and ransomware and are aligned with the goals of my project that aims to utilise bioheuristic techniques for feature selection alone, for optimising ML/DL hyperparameters, and for model architecture for use within ransomware detection.

2.2 Critique of the review

The discussed literature offers valuable insights into the emergence of ransomware historically and the effectiveness of varied approaches to detection, including signature-based, heuristic, behaviour-based, and machine learning approaches. One strength of the studies under scrutiny is their academic merit in terms of technical sophistication and applicability to modern research directions, especially considering a surge in interest in early detection via dynamic and behavioural inspection and the use of bio-inspired algorithms to select features and enhance models.

However, there are some limitations. Much of the research analysed is specific to Android ecosystems, and this could limit the generalisability of findings to systems like Windows systems. In addition, some studies employed publicly available or simulated datasets, and this leads to questions about the applicability and strength of their findings in real-world contexts. While the review outlines numerous detection approaches, there are comparatively fewer studies that evaluate hybrid or ensemble systems combining more than one approach to improve accuracy and robustness.

The study could benefit from a deeper exploration of models' performance and scalability in real-life settings. Few sources covered the issue of the interpretability of machine learning models, an aspect which is ever more crucial in cybersecurity contexts when it is key to comprehend the source of alerts.

In addition to these constraints, the existing synthesis of work presents a good basis to build on in the study, especially in the determination of the strengths and shortcomings of existing methods. The research gaps isolated—cross-platform identification, dataset variation, and interpretability—emphasise areas needing more research and confirm the project's direction on the applicability of machine learning and optimisation algorithms to the area of ransomware detection.

2.3 Summary

This chapter presented a detailed review of existing literature on ransomware detection, tracing the development of ransomware variants and the rising complexity of the methods used to launch attacks. The chapter compared different types of detection approaches, including signature-based, heuristic, behaviour-based, and machine learning approaches, highlighting the strengths and weaknesses associated with each. In addition, the review explored the rising importance of bio-inspired algorithms as a possible pathway to improving the detection system. The gaps in the existing literature identified

were the narrow applicability to different environments, reliance on publicly available datasets, and a lack of hybrid and interpretable models. These findings have significant research methodology consequences and support the use of machine learning and optimisation algorithms to maximise the effectiveness of ransomware detection.

Chapter 3 - Methodology

3.1 Introduction

This chapter outlines the methodological approach taken in the development and implementation of a system for detecting and classifying ransomware as a subset of malware types in general. The ultimate objective is to develop a solid and accurate detection model capable of identifying ransomware in its earliest stages, including new, never-before-seen variants (zero-day). In order to achieve this purpose, a systematically structured testing pipeline is put in place, including data acquisition, preprocessing, feature extraction, model training and evaluation, and optimisation.

Due to the growing complexity and evasive tactics of modern ransomware attacks, signature-based detection methods are now insufficient. Therefore, supervised machine learning and deep learning models are utilised here, complemented by bio-inspired metaheuristic algorithms to enhance performance through automatic hyperparameter selection and feature selection.

The methodology adopted involves using annotated malware datasets with examples of ransomware from datasets like MalMem2022, which also provides behavioural data extracted from memory dumps. Feature engineering and feature extraction methods are used to determine relevant patterns in the datasets. A comparative evaluation is done by using a range of classifiers such as Random Forest, Support Vector Machine (SVM), and Convolutional Neural Networks (CNN). Bio-inspired optimisers like Particle Swarm Optimisation (PSO) are also incorporated to increase the efficiency of the model and the level of correctness in detections.

The models are tested with traditional benchmark performance metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. There is a careful examination of the results to decide on the best technique to achieve real-time ransomware identification. The methodology assures a balanced integration of scientific rigour and practical applicability as well as innovative practices to match the overall goal of improving the development of smart cybersecurity systems.

3.2 Dataset Description and Collection

The dataset used in this work is central to training and evaluating machine learning algorithms to identify ransomware. A dataset supporting annotation and improving the system's ability to discriminate between ransomware and other types of malwares was required. After careful inspection of several sources, MalMem2022 was adopted because it has a vast collection of malware specimens covering ransomware and a strong focus on dumpfile inspection—an area growing increasingly important to obfuscated or evasive threat identification.

The MalMem2022 dataset consists of a total of **58,596** memory dump samples, evenly split between 29,298 benign and 29,298 labelled instances, and covers a range of malware types, such as ransomware, trojans, spyware, etc. The samples are all derived from memory dumps taken while running the malware in a controlled sandbox environment, thus increasing the usefulness of the dataset in identifying distinct runtime behaviours—a critical aspect in the analysis of modern ransomware (Herrera-Silva & Hernández-Álvarez, 2023). The dataset includes static and dynamic feature representations, thus being especially well-suited for the training of supervised learning models.

Rationale for Dataset Selection

The initial datasets considered were options like MOTIF and EMBER. While MOTIF presented high-quality data, it was eliminated on account of having a relatively small sample size limiting its applicability towards generalisable modelling. EMBER presented a larger dataset but used a pre-processed format with fixed vectorised features. This makes it difficult for additional custom derivation of features and exploratory data exploration. MalMem2022 presented strengths in scale, diversity, and unprocessed features and thus allowed greater controllability around preprocessing and feature design as well as greater accessibility in academic settings.

Challenges in Data Collection

One of the key challenges to overcome during this phase was the scarcity of high-quality, harmless data. Samples capable of truly duplicating ransomware behaviours are essential to guarantee models generalisability and reduce false positive rates. However, collecting such data is difficult because of

legal and privacy constraints—data associated with users or systems is commonly encapsulated by proprietary rights, copyright protection, or has legal barriers to public sharing. This limitation is commonly acknowledged in ransomware research and complicates the issue of training balanced detection systems representative of real environmental scenarios.

Despite the limitations discussed above, MalMem2022 proved to be an effective and efficient foundation for this study. The integration of behaviour- and memory-focused data enabled meaningful feature extraction and helped in developing models that can differentiate between ransomware based on execution patterns instead of static signatures.

3.3 Data preprocessing

Preprocessing of the data is required to ensure the validity and generalizability of ML models, particularly in cybersecurity applications when raw data are incomplete, inconsistent, or noisy. In the project, preprocessing methods were established to prepare the MalMem2022 dataset to perform stable training and testing of ransomware detector models.

Data Cleaning

The initial exploration of the dataset did not reveal many issues such as missing values and redundant features. Missing values were addressed using imputation or feature elimination according to the feature importance. Redundant records were deleted so each malware sample would have a unique contribution towards the model.

Feature selection and reduction

Since the features extracted from the memory dumps are of high dimension, feature selection was conducted to reduce noise and computation overhead. Domain knowledge and statistical methods (like correlation analysis and variance thresholding) were used to choose the most discriminative features and keep only them. A bio-inspired search methodology, Particle Swarm Optimization (PSO), was also used to look for the optimum subset of features as a way of improving the classification rate by eliminating redundancy.

Data Normalization

Since the dataset contained features of varying scale (e.g. frequency data and binary indicators), features were normalised to a uniform range of values between 0 and 1 using min-max scaling. This ensured the learning process was not controlled by a particular feature, and in particular by magnitude-sensitive features like neural networks and SVMs.

Label Encoding

The target classes (benign, ransomware, spyware, trojan) were numerically encoded to facilitate supervised learning. In the case of binary-classification experiments, the ransomware samples were labeled as 1 and all other malware or normal as 0. In multi-class experiments, a one-hot encoding was adopted.

Data Splitting : The preprocessed data were split into training data and test data with a split of 70:30 with stratification to retain the class distribution. k-fold cross-validation (with k=5) on training data was used to reduce variance and to prevent overfitting to add extra robustness.

3.4 Feature Extraction and Engineering

Feature extraction is a fundamental process of transforming raw data into structured inputs to feed machine learning models. In ransomware detection, the quality and relevance of features obtained using feature extraction have a direct impact on model performance. The work involves extracting behavioral features from the memory dump data available in the MalMem2022 dataset and representing actual runtime behavior of malicious software.

3.4.1 Methods of Extraction

The following were extracted and engineered:

- API Call Frequencies: Frequency of prominent API calls (e.g. CreateFile, WriteFile, EncryptFile) typically invoked by ransomware while conducting its encrypting operations.

- High entropy files are an indicator of encrypted or compressed data and a telltale sign of a ransomware payload.
- Access patterns to particular areas of memory were examined as part of attempts to look for suspicious in-memory encryptions.
- File System Activities: The bulk creation, deletion, or renaming operations were used as indicative features.
- Process Creation Trees: Patterns of child process spawning (typically used to avoid detection) were tracked.

These were either already part of the MalMem2022 dataset or were generated using open-source libraries and custom scripts.



Figure 1

3.4.2 Domain Relevance

These characteristics are derived from patterns of behaviour typical of ransomware. Ransomware tends to encrypt a user's files to produce high-entropy and specific patterns of API calls. The monitoring of such behaviours allows the detection model to differentiate ransomware from other malware or legitimate programs even in zero-day cases.

3.4.3 Dimensionality Reduction (Optional)

To increase the performance of the model and reduce the level of computation required, Principal Component Analysis (PCA) was explored as a dimension reduction technique. Bio-inspired feature selection using Particle Swarm Optimisation (PSO) was eventually used in the ultimate model to pick the most discriminant features according to the extent they contributed to model precision increase.

3.4.4 Tools and Technologies

- Scikit-learn – selection, transformation and PCA
- PESTudio - static executable analysis (to perform early-stage testing)
- Cuckoo Sandbox - dynamic behavioral analysis (where appropriate)
- Custom Python scripts – to compute the entropy, monitor API calls and feature extraction logic

3.5 Machine learning algorithms

3.5.1 Overview of algorithms

The primary aim of this research effort is the development of a machine learning-based framework that efficiently classifies ransomware under different categories of malware and effectively discriminates between dangerous and innocuous software as required. Given the covert nature inherent in advanced ransomware attacks, the detection framework needs to effectively identify behavioural patterns pursuant to the execution of ransomware that include behaviours associated with encryption actions, API calls, and memory access operations.

To achieve this, a range of machine learning algorithms were explored, focusing on those with proven effectiveness in malware and anomaly detection tasks. The aim was to select models that offered a balance between classification accuracy, model interpretability, training efficiency, and generalisation to unseen samples — especially important in identifying zero-day ransomware.

The first step in the selection process was a review of a variety of supervised learning algorithms, including traditional machine learning models and methods based on neural networks. Every algorithm was evaluated based on the following criteria:

- Ability to handle large-scale high-dimensional behavioural features.
- Versatility across binary and multi-class classification platforms.
- Performance in prior ransomware or malware detection studies
- Enabling the integration of bio-inspired optimization techniques. Computational efficiency for real-time application

The following comparison table shows the different algorithms that were considered:

Algorithm	Type	Strengths	Limitations	Sustainability for ransomware detection
Random Forest (RF)	Ensemble	Robust, handles high-dimensional data	Slower with large feature sets	Strong baseline classifier
Support Vector Machine(SVM)	Supervised	Performs well on high-dimensional data	Sensitive to feature scaling, tuning needed	Effective for binary classification
K-Nearest Neighbour(KNN)	Lazy	Simple, interpretable	Poor with noisy/ high-dim data	Not ideal for malware detection
Logistic Regression	Linear	Fast, interpretable	Limited with nonlinear decision boundaries	Too simplistic
CNN (Deep Learning)	Neural Net	Learns complex patterns from raw data	Needs lots of data + compute	Optional for advanced experiments

Table 2.1

3.5.2 Final algorithm choices and justification

Following consideration of many models, the following models were selected for utilisation, as they offered a balance of applicability within the domain, interpretability, and performance:

Random Forest (RF): A collection of trees that is appropriate for structured and tabular data capable of handling categorical and continuous features. RFs perform well when handling overfitting and have performed successfully in the area of malware classification (Biau & Scornet, 2016).

Support Vector Machine (SVM): A powerful classification technique that aims for the largest margin between classes in high-dimensional spaces. SVMs naturally suit anomaly detection and have already proven effective in the realm of ransomware detection (Aldhafferi, 2024).

(Optional) Convolutional Neural Networks (CNN): CNNs have primarily been used with image-based data but have already proven useful when memory dumps or binaries are translated into image-like format. This facilitates deep models in identifying complex patterns of ransomware activity (Brosolo et al., 2025).

3.5.3 Optimisation Using Bio-Inspired Algorithms

To further enhance performance, Particle Swarm Optimisation (PSO) is used to automate hyperparameter tuning (e.g., number of trees in RF, kernel type in SVM). PSO simulates the movement of particles in a search space to iteratively converge on optimal values. This improves model accuracy while reducing the need for manual tuning (El-Taweel et al., 2024).

In the figure below, there is a diagram for the phases of the PSO algorithm. The five stages are: 1. Initialise particles, 2. Evaluate fitness, 3. Train ML model with selected features. 4. Update pBest and gBest, 5. Update particle velocity and position.

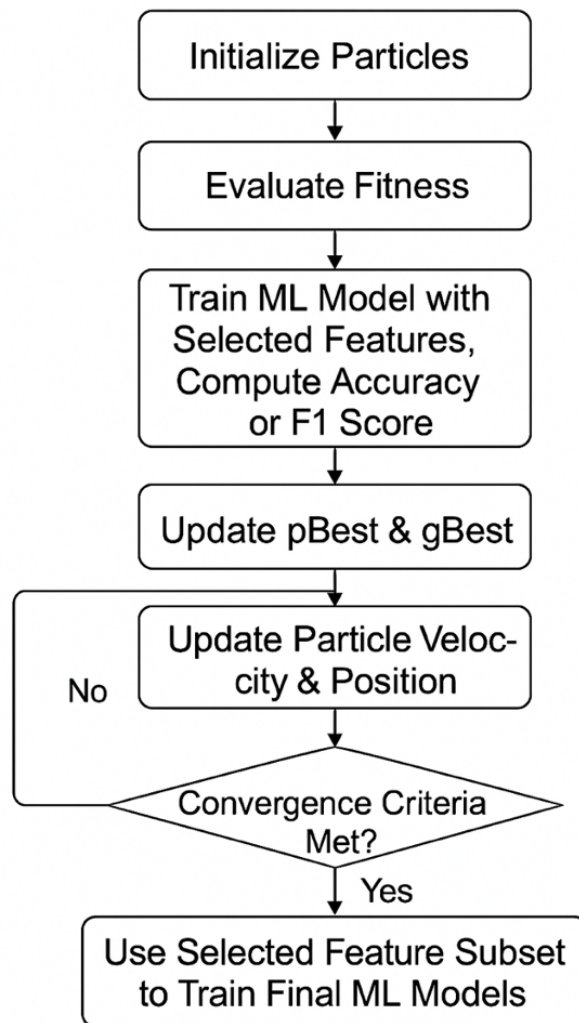


Figure 2

3.6 Breakdown of the process

Particle Initialisation:

Each particle in the swarm represents a potential subset of selected features from the entire space of features. The particles are initialised with random binary values such that a '1' represents the availability of a feature and a '0' represents its non-availability. Random initial value allocation that is inherent in its nature increases the ability of the algorithm to explore a wide range of alternative feature combinations in the beginning of the process.

Fitness Evaluation:

The quality of each of the particles is evaluated by training a machine learning classifier using the specific subset of features that it represents. The performance of the system is evaluated in terms of its performance measures against the accuracy or F1-score obtained from the validation set. The approach ensures subsets that show the best classification performance are ranked more highly.

Revaluation of Personal and Global Best:

Every particle keeps an eye on its best current position, known as the personal best (pBest), calculated based on previous fitness evaluations. Moreover, the particle swarm keeps a track of the best solution across the whole population termed as the global best (gBest). These best positions act as a reference point for the movement of the particles in the search space.

Re-evaluation of Position and Velocity:

Each particle updates its velocity and feature selection position by considering both its pBest and the gBest. The new position is computed using the standard PSO formula, and binary decisions (select/deselect features) are made using a thresholding mechanism (e.g. sigmoid function and binary rounding).

Decision-making and termination of employment:

The process is executed iteratively for a fixed number of iterations until the process converges as manifested by the negligible improvement in gBest. The final result is the set of optimum features associated with the particle with the best performance that is then used as the input for fine-tuning the last stage machine learning models.

For the hyperparameter optimisation, standard PSO was employed. Conventional GA mutation operators, such as bit flip, swap, and inversion, were not used because the swarm dynamics did not incorporate crossover or mutation operations.

Model Training and Evaluation

To ensure reliable performance estimation and generalisability, the database passed through an 80/20 split into a training subset and a testing subset. The training subset was used to train the machine learning models, while the testing subset was reserved for final evaluation. Stratified sampling was used in an effort to maintain the class balance between ransomware and non-ransomware samples.

In addition to splitting data into training and testing sets, a 5-fold cross-validation process was applied to the training set to reduce variance in the model's evaluation and avoid overfitting. This approach allowed each model to be trained and validated on different sections of the dataset, thus providing a more robust estimate of its ability to generalise.

For the deep learning experiments using Convolutional Neural Networks (CNNs), the training protocol will be defined with *(due to time constraints, deep learning could be a theoretical piece and not a practical endeavour in this project)*:

- Batch size: 32
 - Epochs: 20–50 (adjusted based on early stopping criteria)
 - Optimiser: Adam
 - Loss function: Binary cross-entropy (for binary classification)
- Machine learning and deep learning hyperparameters were optimised by the use of Particle Swarm Optimisation (PSO), focusing on parameters such as the number of estimators in Random Forest, the kernel and the C-value of Support Vector Machines (SVM), and the feature selection subset size.

All the training protocols will be performed using Python with Scikit-learn for standard machine learning and TensorFlow/Keras for deep learning tasks. The performance of the models was rigorously evaluated using standard metrics mentioned in the next section.

3.7 Evaluation criteria

To determine the effectiveness of the suggested detection models, several evaluation measures were used to provide a comprehensive view of their classification performance, given the serious real-world implications involved in the correct identification of ransomware.

Accuracy

Accuracy measures the proportion of accurate predictions out of the total predictions made. While it provides a general measure of the effectiveness of a model, it might fail to capture performance in the presence of imbalanced data. For instance, in situations where ransomware incidents are rare compared to other malware or benign files, a model can achieve high accuracy by always predicting the dominant class. As such, while accuracy is a valuable measure, it should not be the sole metric for model performance evaluation.

Precision, Recall, and F1-Score

Precision refers to the rate of predicted ransomware attacks that match true ransomware attacks. High precision implies a low rate of false positives, a crucial consideration in avoiding unnecessary alarms or isolation of innocuous files.

Recall, more commonly known as sensitivity, is the proportion of correctly identified true ransomware attacks. In this context, high recall is necessary because the consequence of missing a ransomware threat (a false negative) with regard to the system's integrity is severe.

F1-Score is the harmonic mean of precision and recall that presents an even analysis in situations involving both the high cost of false positives and false negatives. It is particularly helpful when there is class imbalance and when there is a trade-off between precision and recall.

Confusion Matrix

The confusion matrix gives a thorough analysis of each class of prediction that enables the identification of multiple error categories:

- True Positives (TP): Ransomware correctly detected
- False Positives (FP): Benign or non-ransomware misclassified as ransomware

- True Negatives (TN): Non-ransomware instances correctly identified. False Negatives (FN): Ransomware samples incorrectly classified as non-ransomware.

In ransomware detection, the most dangerous part is false negatives since they represent threats that are malicious and not detected. The confusion matrix allows for a deeper analysis of such important misclassifications, thus informing both model selection and further improvements.

ROC-AUC Curve

The performance metric called ROC-AUC (Receiver Operating Characteristic – Area Under Curve) measures the ability of a model to distinguish between classes at various threshold values. AUC values close to 1.0 indicate a strong separation power.

Receiver-Operating Curve

The Receiver Operating Characteristic (ROC) curve traces the model's ability to differentiate the ransomware class from the rest at different threshold levels. The Area Under the Curve (AUC) quantifies the separation — with values closer to 1.0 being reflective of greater discriminatory power.

For the task at hand, the ROC-AUC metric was most useful in comparing classifiers like Random Forest and SVM since it provides a threshold-independent metric of performance that is optimal when models output probability scores.

Chapter 4 - Implementation

4.1 Baseline model development

The reliability of the dataset and the appropriateness of the model were first confirmed during the initial stage of binary classification. In this stage, files were labelled as either malicious or benign through simple manipulation of the Category column. This allowed for testing of multiple machine learning classifiers (Random Forest, KNN, SVM) in a more manageable setup as an introduction to the final task of ransomware family differentiation. In addition, this process ensured the pipeline of the data, preprocessing methods, and model structure were strong enough and ready for a more complex classification task.

Preliminary findings:

Random Forest, K-Nearest Neighbours, and Support Vector Machine models recorded an overall accuracy of higher than 99%, as validated using 5-fold cross-validation methods. Specifically, the highest mean accuracy of 0.9987 was recorded for the Random Forest model with precision and recall values near 1.00, which signified an excellent ability to separate benign from malicious samples. The classification reports support this claim:

```
=== Classification Report ===
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	8790
1	1.00	1.00	1.00	8789
accuracy			1.00	17579
macro avg	1.00	1.00	1.00	17579
weighted avg	1.00	1.00	1.00	17579

Figure Y

```
=== Classification Report (SVM) ===
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	8790
1	1.00	1.00	1.00	8789
accuracy			1.00	17579
macro avg	1.00	1.00	1.00	17579
weighted avg	1.00	1.00	1.00	17579

Figure Z

```
=== Classification Report (KNN) ===
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	8790
1	1.00	1.00	1.00	8789
accuracy			1.00	17579
macro avg	1.00	1.00	1.00	17579
weighted avg	1.00	1.00	1.00	17579

Figure X

The binary classification step served as an important baseline for measuring the ability of traditional machine learning models in classifying benign and malicious samples. Models such as Random Forest, KNN, and SVM were put through training and evaluation, with each achieving mean accuracy rates above 99%, along with almost perfect precision and recall scores. These results validated the quality of the dataset and confirmed that the extracted memory-based features had strong discriminatory power.

Appendix A includes the overall implementation code used, as well as the evaluation of the models along with respective confusion matrices. The project then proceeded towards its main goal: ransomware classification into respective families

4.2 Data preparation

The dataset used in the current research is MalMem2022 dataset, an openly available memory dump dataset specially developed for malware classification tasks. This dataset consists of behavioural features from memory extracted from benign as well as malicious samples of ransomware, spyware, and trojans. Each sample consists of numerical features like process counts, thread counts, handle counts, and DLL counts during the execution time, which in combination capture the runtime behaviour of the file.

The dataset initially had over 22,000 samples. However, since the project's interest is only in the detection of ransomware and classification of its families, all the entries unrelated to ransomware needed to be excluded.

This was achieved by filtering on the dataset using the Category column where ransomware samples were assigned with unique family names (such as ransomware-maze, ransomware-ako). The Class column, which only classified the file as “Malware” or “Benign” was not specific enough to fulfill the objectives of this research.

Filtering step:

```
# Load dataset
df_full = pd.read_csv("MalMem2022.csv")

# Clean the 'Category' column to avoid casing/spacing issues
df_full['Category'] = df_full['Category'].str.strip().str.lower()

# Filter ransomware samples (those that start with 'ransomware')
ransom_df = df_full[df_full['Category'].str.startswith('ransomware')].copy()
```

Figure 1.1

The filtration process yielded a dataset of 9,791 ransomware samples that fall into five distinct families of ransomware:

- Maze
- Ako
- Pysa
- Conti
- Shade

Each row in the filtered data is a unique ransomware sample, along with its corresponding behavioural feature values and labelling as belonging to a family.

Initial preprocessing

Attributes that were considered unnecessary, like the Filename and the general Class identifier, were removed because they were not useful for the purposes of classification. The attributes that were kept were exclusively numeric runtime features and were retained for model training.

```
# --- Drop unused columns ---
ransom_df = ransom_df.drop(columns=['Class', 'Category', 'Filename'])
```

Column removal step:

Figure 1.2

To prepare target labels for learning, the ransomware family text-label names in the Category were assigned a numerical label.

```
# --- Encode ransomware families ---
label_encoder = LabelEncoder()
ransom_df['family_label'] = label_encoder.fit_transform(ransom_df['Category'])
```

Figure 1.3

It was the foundation for all subsequent model testing and training on this well-structured and clean dataset.

4.2.1 Feature selection and preprocessing

To prepare the ransomware classification dataset for model training, several preprocessing steps were utilized:

Feature Retention

Non-numerical columns that contained no information were dropped. Specifically, the following columns were dropped:

- *Category*: It was systematically classified using numerical designations (family_label).
- *Filname*: Not suitable for classification.
- *Class*: Duplicated higher-level labels (e.g., "Malware", "Benign").

All remaining columns represented behavioural or memory-based features extracted from memory forensic tools such as Volatility.

Label encoding

The ransomware family names (e.g., 'ransomware-maze', 'ransomware-conti') were encoded using LabelEncoder from scikit-learn: This made the target variable compatible with scikit-learn classifiers.

Train/ test split

The data was split into 70% training and 30% testing using `train_test_split()`, stratified by ransomware family to preserve label distribution:

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)
```

Figure 1.4

Feature normalisation

To ensure equal feature importance and improve convergence of certain models (e.g., SVM, KNN), `StandardScaler` was applied:

```
# Normalize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Figure 1.5

This transform features to have a mean of 0 and a standard deviation of 1.

Standardization was especially important for the distance-based models like KNN and margin-based classifiers like SVM, as large-magnitude features can dominate otherwise.

Before filtering –image shows the first 5 rows from the full dataset.

	pslist.nproc	pslist.nppid	pslist.avg_threads	pslist.nprocs64bit	pslist.
0	45	17	10.555556	0	
1	47	19	11.531915	0	
2	40	14	14.725000	0	
3	32	13	13.500000	0	
4	42	16	11.452381	0	

5 rows × 58 columns

Figure 2.1

After filtering – image shows the last 5 rows of the ransomware-specific filtered dataset.

	pslist.nproc	pslist.nppid	pslist.avg_threads	pslist.nprocs64bit	psli
29298	39	15	11.384615	0	
29299	37	15	10.135135	0	
29300	39	15	10.666667	0	
29301	41	16	10.121951	0	
29302	41	16	9.731707	0	
5 rows x 58 columns					

Figure 2.2

4.2.2 Random Forest classifier

To establish a base standard of classification, the Random Forest classifier was used due to its stability, ability to deal with imbalanced datasets, and ease of interpretation. In being an ensemble approach built on decision trees, Random Forest exhibits lower susceptibility to overfitting and possesses inbuilt feature importance measures, making it of particular use for exploratory analysis of this dataset.

Configuration:

The classifier was configured with the following parameters:

- *n_estimators*=100: Specifies the number of decision trees in the forest.
- *max_depth*=None: Allows trees to expand fully (default).
- *random_state*=42: Ensures reproducibility.

Training process:

Following the preprocessing and partitioning of the dataset into training and test subsets in the ratio 70/30, the model was trained on only the ransomware samples with the target of predicting the encoded ransomware family (*family_label*).

The performance of the Random Forest algorithm was impressive, showing variation in between various ransomware families—indicating its ability to distinguish between certain families with more confidence compared to others.

Evaluation:

The model's performance was assessed using:

- **Classification Report:** Included precision, recall, F1-score, and support for each ransomware family.

=== Classification Report (Random Forest) ===				
	precision	recall	f1-score	support
ransomware-ako	0.52	0.54	0.53	600
ransomware-conti	0.53	0.54	0.53	597
ransomware-maze	0.58	0.62	0.60	587
ransomware-pysa	0.52	0.47	0.50	515
ransomware-shade	0.64	0.61	0.63	639
accuracy			0.56	2938
macro avg	0.56	0.56	0.56	2938
weighted avg	0.56	0.56	0.56	2938

Figure 2.3

Per-class performance:

Ransomware-Maze and Ransomware-Shade had the highest F1-scores at 0.60 and 0.63, respectively, reflecting the best ability of the model to differentiate between them accurately.

Ransomware-Pysa had the lowest F1-measure of 0.50, signifying that it was more difficult to distinguish from other families due to shared behavioural attributes.

Precision and recall scores remained relatively balanced across classes, which shows the model is not overly biased toward any single ransomware type.

General accuracy

The model reached an accuracy of **0.56**, which means 56% of the test samples were classified accurately. While the score is fairly low, it is useful for comparison against other models.

Confusion Matrix: A heatmap was generated to visualize class-specific prediction performance.

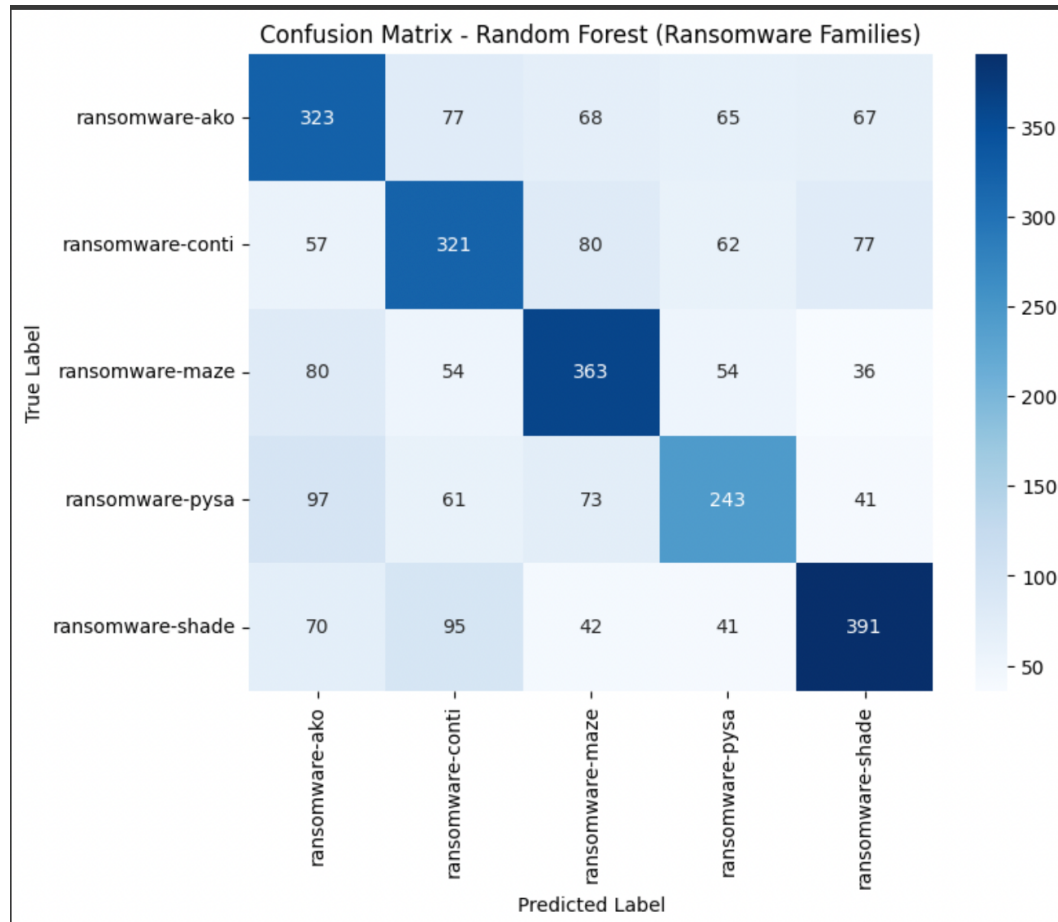


Figure 3.1

4.2.3 K-nearest neighbours (KNN)

K-Nearest Neighbours (KNN) was chosen since it is straightforward and performs well in classification problems when the decision boundaries are not linear. KNN is a distance-based classifier; thus, it relies heavily on the distribution and scaling of the features. This makes the use of StandardScaler prior to KNN a crucial process within data preparation.

Changing Hyperparameters

To enhance the way the model functions, we accommodated changes in the number of neighbours (k). We tested k between 1 and 10 through cross-validation. We plotted the accuracy values, and what we got was the best mean accuracy from $k=1$. This indicates that only one nearby neighbour had sufficient information for classification here.

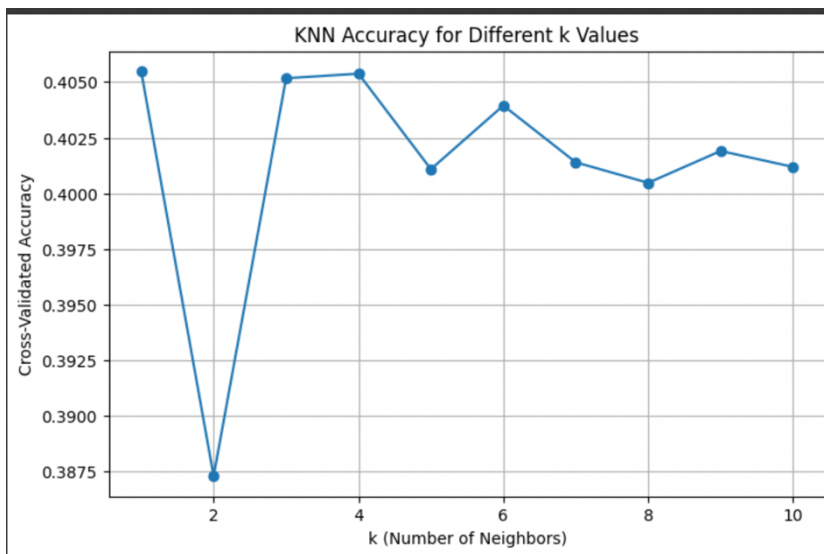


Figure 3.2

The model achieved a cross-validation accuracy of over 99% with $k=1$, demonstrating excellent performance during the binary classification stage (malicious vs. benign).

Final Training and Testing

Selecting $k=1$, the model was then trained on the ransomware family dataset. Prediction was made on the test set, and performance was evaluated using classification report and confusion matrix.

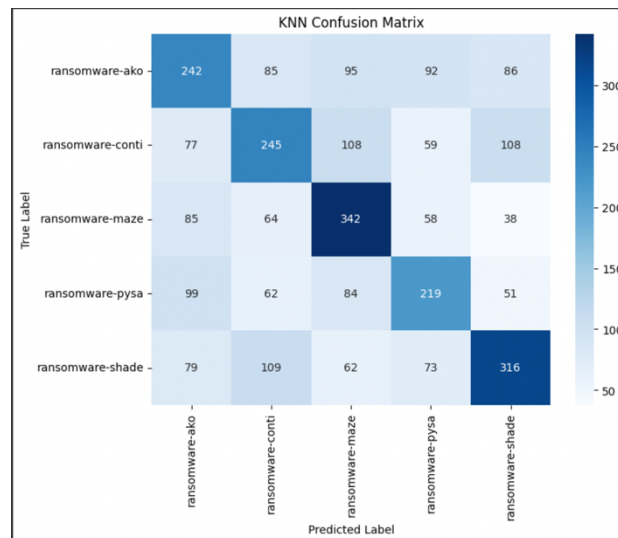


Figure 3.3

```
=== Classification Report (KNN) ===
```

	precision	recall	f1-score	support
ransomware-ako	0.42	0.40	0.41	600
ransomware-conti	0.43	0.41	0.42	597
ransomware-maze	0.49	0.58	0.54	587
ransomware-pysa	0.44	0.43	0.43	515
ransomware-shade	0.53	0.49	0.51	639
accuracy			0.46	2938
macro avg	0.46	0.46	0.46	2938
weighted avg	0.46	0.46	0.46	2938

Figure 3.4

This was a mixed performance, especially in identifying certain ransomware families, though with less accuracy in comparison to Random Forest. This was to be anticipated, as KNN is susceptible to high-dimensional feature space as well as overlapped classes.

- The model struggled to distinguish some classes due to overlapping behavioural patterns in feature space.

- Model has misclassifications, especially between ako, conti, and pysa, indicating a need for more sophisticated decision boundaries than KNN can provide with high-dimensional malware data.

Strengths and weaknesses

- Strengths: Quick to deploy and simple; worked well in binary classification.
- Weaknesses: Did not perform well on multi-class classification due to imbalance and overlap of classes.
- *While KNN offered exceptional performance in early stages of analysis, its limitations became more apparent during ransomware family classification, reinforcing the need for more sophisticated models.*

4.2.4 Support Vector Machine (SVM) – Baseline

The SVM model was chosen for its effectiveness in high-dimensional spaces and its ability to define optimal separating hyperplanes. Initially, a baseline model was trained using:

- Kernel: 'rbf'
- C: 1.0
- Gamma: 'scale'

The baseline model showed poor performance on ransomware classification, with:

- Accuracy: ~0.28
- Macro F1-Score: ~0.17
- Severe class imbalance and overlapping distributions contributed to low recall for some families (e.g., ransomware-conti and ransomware-maze with F1-scores of 0.02 and 0.00, respectively).

```

=== Classification Report (SVM) ===

```

	precision	recall	f1-score	support
ransomware-ako	0.51	0.23	0.32	600
ransomware-conti	0.56	0.01	0.02	597
ransomware-maze	0.00	0.00	0.00	587
ransomware-pysa	0.77	0.07	0.13	515
ransomware-shade	0.24	1.00	0.39	639
accuracy			0.28	2938
macro avg	0.42	0.26	0.17	2938
weighted avg	0.41	0.28	0.18	2938

Figure 3.5

Hyperparameter Optimisation with PSO

Then describe how PSO was used:

SVM with PSO (Particle Swarm Optimisation)

To improve the SVM's performance, particle swarm optimisation was used to tune two critical hyperparameters:

- C (regularisation) — controls the trade-off between margin maximisation and misclassification.
- Gamma — defines how far the influence of a single training point reaches.

Using the *pyswarms* library, PSO was run for 10 iterations, optimising over a search space of:

- C: [0.1, 100]
- Gamma: [0.001, 1]

The best parameters found were:

- C = 44.45
- gamma = 0.052

The optimised SVM achieved significantly better results:

```
pyswarms.single.global_best: 100%|██████████| 10/10, best_cost=0.529
2025-05-16 02:34:03,148 - pyswarms.single.global_best - INFO - Optimization
=== Confusion Matrix (SVM + PSO) ===
[[318 102  56  57  67]
 [ 74 303  89  30 101]
 [118  91 315  24  39]
 [135  64  90 173  53]
 [ 81 106  48  27 377]]

=== Classification Report (SVM + PSO) ===
```

	precision	recall	f1-score	support
ransomware-ako	0.44	0.53	0.48	600
ransomware-conti	0.45	0.51	0.48	597
ransomware-maze	0.53	0.54	0.53	587
ransomware-pysa	0.56	0.34	0.42	515
ransomware-shade	0.59	0.59	0.59	639
accuracy			0.51	2938
macro avg	0.51	0.50	0.50	2938
weighted avg	0.51	0.51	0.50	2938

Figure 3.6

This improvement indicates that tuning helped SVM learn better class boundaries for the ransomware families.

4.3 Deep learning constraints

While deep learning has gained prominence in cybersecurity applications, it was not pursued in this project due to several practical considerations. Firstly, deep learning models typically require large and diverse datasets to perform effectively, and the size and structure of the MalMem2022 dataset were more appropriate for classical machine learning approaches. Additionally, deep learning models involve significant computational overhead and often require specialised hardware such as GPUs, which was beyond the scope and available resources of this project. Interpretability was another key factor; classical models like Random Forest and SVM provide clearer insight into feature importance and decision boundaries, which is valuable for understanding ransomware behaviour. Most importantly, the focus of this project was on enhancing classification performance through bio-inspired optimisation techniques like Particle Swarm Optimisation (PSO), which proved highly effective when applied to traditional algorithms. Given the project's timeframe and objectives, prioritising efficient and explainable models over complex deep learning architectures was both strategic and justified.

4.4 Conclusion

The Random Forest algorithm served as a solid benchmark, showcasing stable performance and resilience against class imbalance. The k-Nearest Neighbours (KNN) algorithm was explored thanks to its simplicity and interpretability; however, it was faced with difficulties when dealing with high-dimensional and overlapping ransomware datasets. The Support Vector Machine (SVM) initially showed poor performance with its default parameters; however, its performance drastically improved through the use of Particle Swarm Optimisation (PSO), thus illustrating the benefits of using bio-inspired approaches for hyperparameter optimisation. Overall, this stage set a solid ground for evaluating ransomware detection approaches and prepared the models for deployment in a real-time, web-based environment.

4.5 Flask Web Application Integration – Introduction

To promote both interactivity and usability of the ransomware detection system, an operational web application was built using Flask. The interface allows the submission and exploration of malware sample data in real time, thus proving the real-world applicability of the trained machine learning models. The selection of using Flask was prompted by its ease of use, flexibility, and compatibility with Python-orientated machine learning packages. By integrating the PSO-optimised SVM model into the web application, the system is then capable of categorising ransomware into specific families and delivering users meaningful predictions. This implementation step bridges the gap between experimental machine learning practices and an actual cybersecurity tool in line with the project's aspiration of providing an efficient detection platform for real-world purposes.

4.5.1 Flask system structure

To support real-time functionality of the ransomware detector, a modular and flexible architectural design was created for the Flask application framework. This segregation of functions allows for increased efficiency in terms of maintenance, simplifies debugging, and allows direct inclusion of machine learning functionality.

The basic structure is comprised of the following elements:

- `app.py`: This is the main component of the application, which sets up the Flask framework, handles routing processes, and coordinates the critical business logic, including user authentication and predictive modelling.
- `templates/`: This folder includes every HTML template used by the application for rendering purposes. They include user authentication, sign-up processes, a dashboard user interface, and the component for a ransomware detector.
- `static/`: This folder includes customised CSS and Bootstrap components for the visual styling of the user interface, thus ensuring responsiveness and a modern look.
- `uploads/`: an interim directory for safe storage of user-uploaded .csv files for the analysis period
- `model_training/`: Holds the trained machine learning model (`svm_model_pso.pkl`) that was trained offline and loaded at run-time afterwards.

- data/ : Used during the model training process to keep raw datasets (e.g. MalMem2022.csv).
instance/: Holds the SQLite database file utilized for user registration and authentication purposes.

```
flask_app/
├── app.py                # Main Flask application
├── database.db           # SQLite database (auto-created)
├── templates/           # HTML templates for UI
│   ├── dashboard.html
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   ├── ransomware_detection.html
│   └── simulation.html
├── static/              # CSS, JS, or other assets
│   └── style.css
├── model_training/      # Trained ML model stored here
│   └── svm_model_pso.pkl
├── data/                # Original dataset used for training
│   └── MalMem2022.csv
├── uploads/             # Temporary folder for uploaded test files
│   └── test_sample_corrected.csv # Example test input
└── instance/            # Flask-specific config folder
    └── (Holds SQLite DB, config if any)
```

Figure 4.1

This architectural design is based on Flask's optimal methods and integrates into operational processes for instant ransomware predictions. The user-generated data, which includes uploaded feature files, is managed by a uniform and secure back-end system, while predictive results are communicated using an uncomplicated and user-friendly front-end interface.

Trained model ready for integration

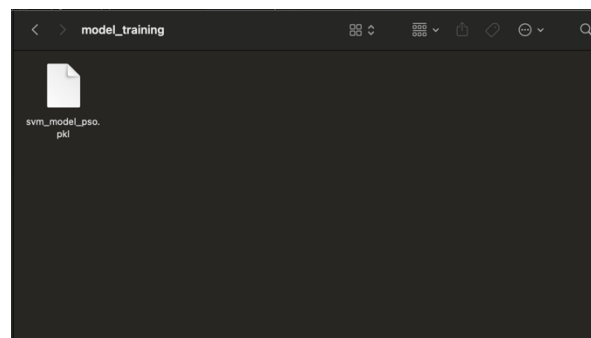


Figure 4.2

4.5.2 User-authentication system

To ensure safe and individualised access, the online application is equipped with an end-user authentication mechanism that is created using Flask-Bcrypt and Flask-Login. During sign-up, users provide an e-mail address and password that is hashed using the Bcrypt encryption method immediately and saved to the database, never revealing sensitive credentials in plain text form to unauthorised parties.

User session handling and route protection are facilitated by Flask-Login, which handles users' sessions and gives access to pages that require it, namely the dashboard and ransomware detector. Successful login users get directed to their dashboard, while unauthorised users get directed to login pages.

The authentication is supported by a User model that is constructed using SQLite to represent the schema of the database. The users are uniquely identified by an email address, and passwords are

```
class User(db.Model, UserMixin):
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(150), unique=True, nullable=False)
    password = db.Column(db.String(150), nullable=False)
```

hashed and saved safely in the database.

Figure 5.1

In line with current best practices in web security, this approach sets up future support for multiple users within successive versions of the system.

Login page

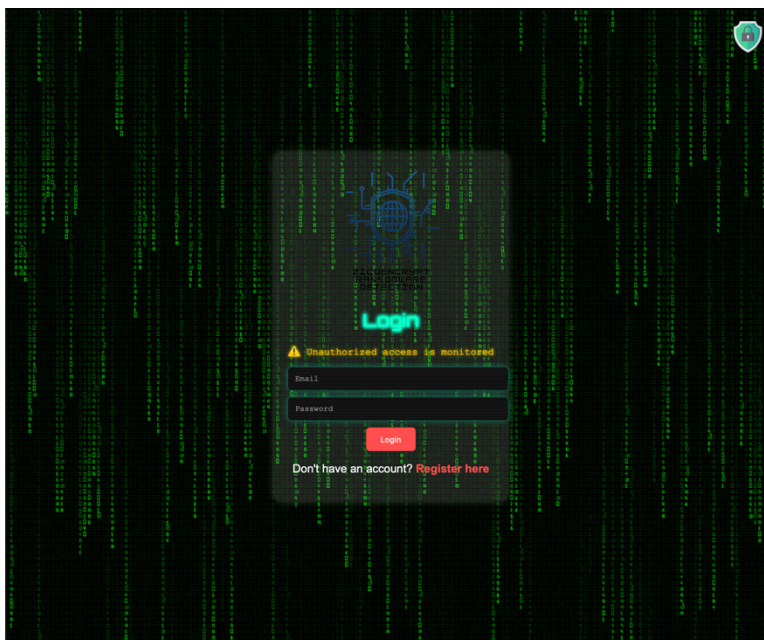


Figure 5.2

Register page

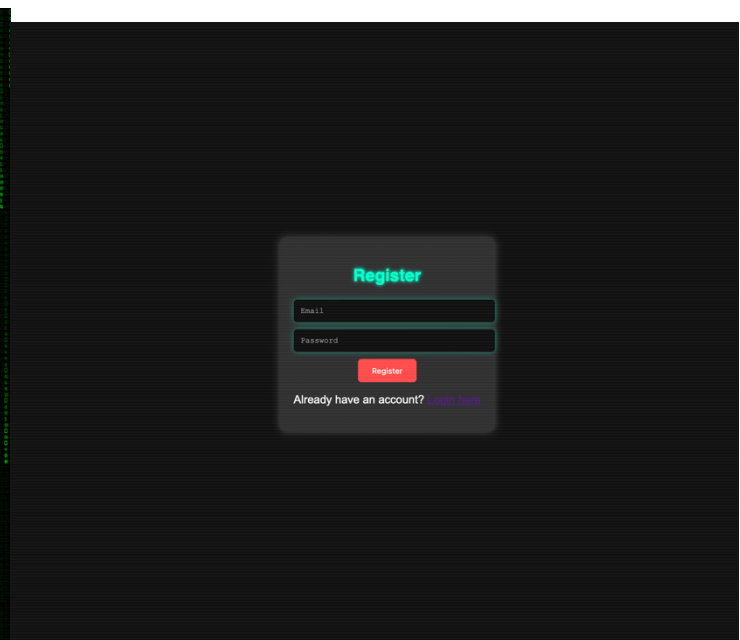


Figure 5.3

4.5.3 Dashboard and navigation interface

Upon successful log-in, the user is routed to the custom dashboard, the nerve centre of the ransomware detection tool. The dashboard is contemporary with the incorporation of sidebar-based design, ensuring it is straightforward and easy to manoeuvre the application features. Ease of use has been given preference, with the dashboard bringing together critical features such as access into the ransomware detection tool, ransomware awareness, the simulation module, and log-out into an easily accessible manner in the sidebar.

The left-hand sidebar displays the user's email address at the very top and uses descriptive icons for all the navigational links in order to enable visual appeal and end-user interactivity. The design has an intuitive feel, with all the important features just a click away. Context-specific data — such as the welcome note when the page is being opened for the very first time, or the scan result summary after conducting a ransomware scan — is displayed in the centre section of the page.

Dashboard

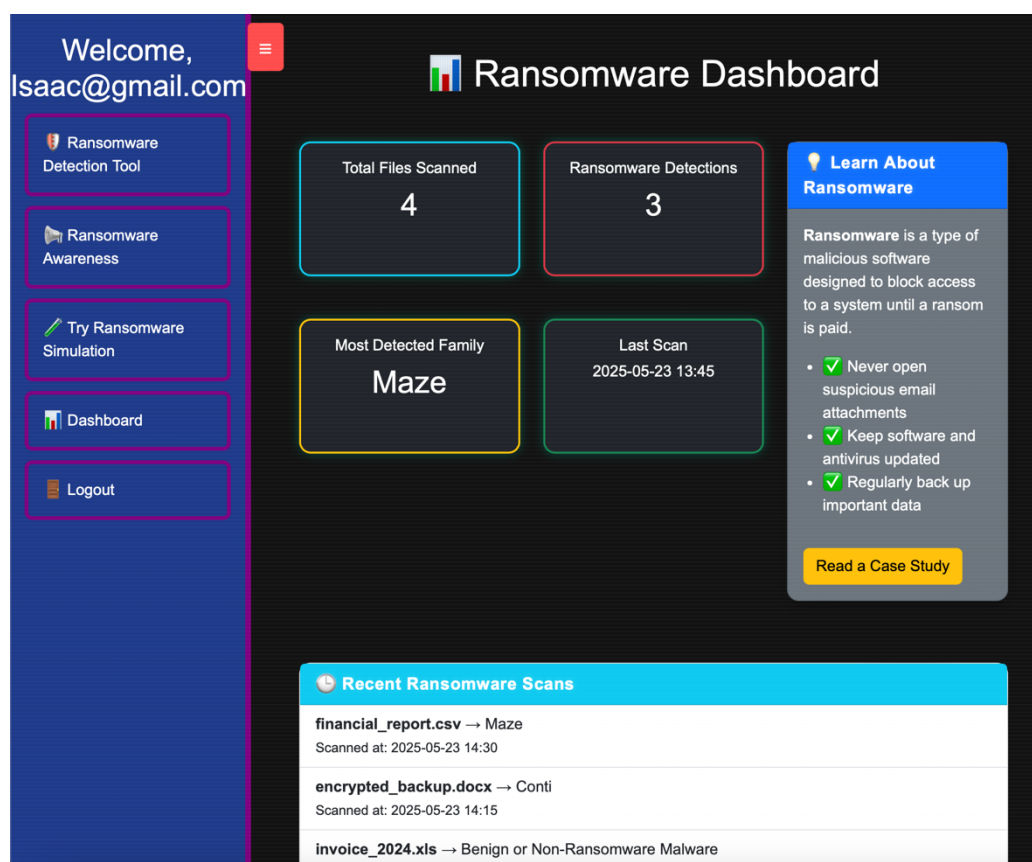


Figure 5.4

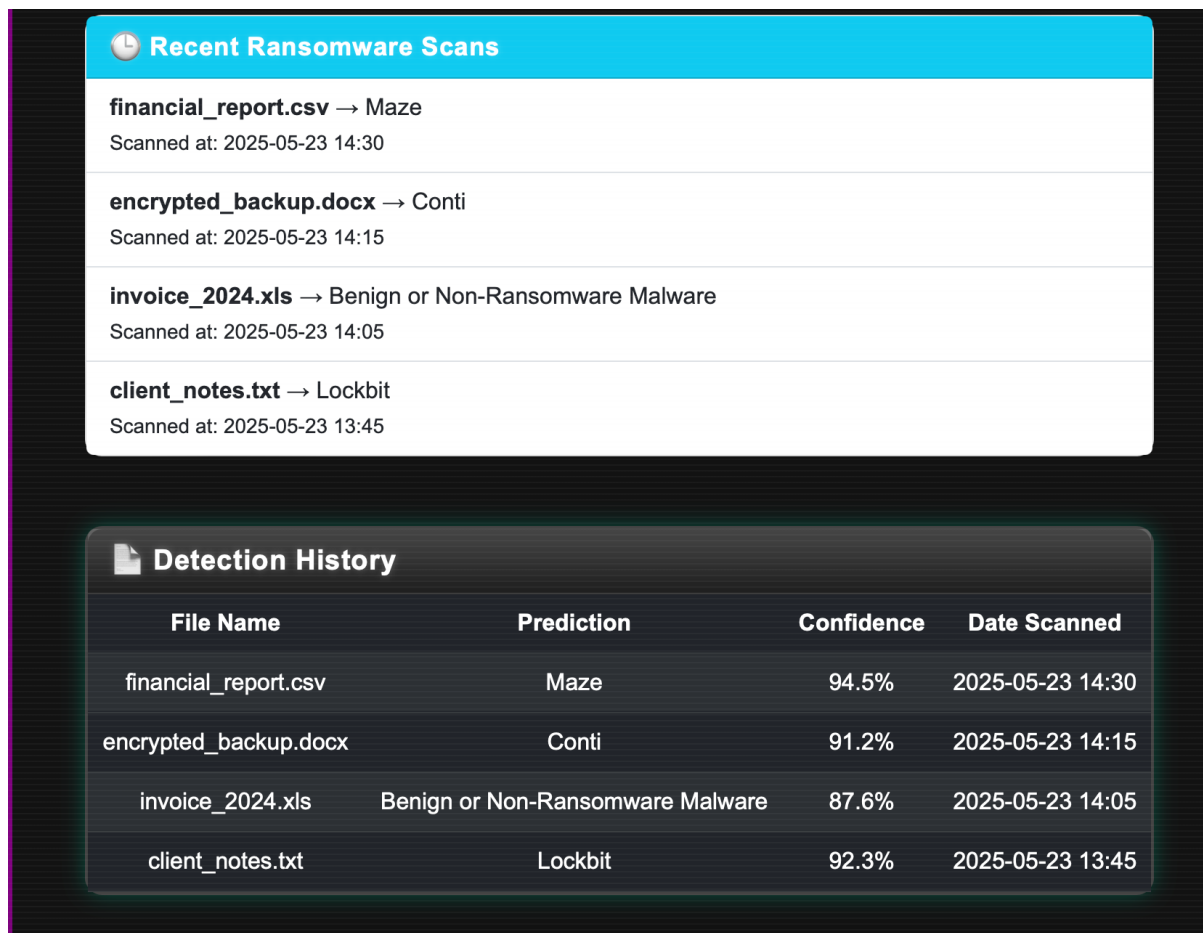


Figure 5.4.1

Description of each link:

- Ransomware Detection Tool – directs the user to upload a file for malware analysis.
- Ransomware Awareness – (optional placeholder) can be used to show info about threats.
- Try Ransomware Simulation – opens a page to explore simulated attacks.
- Dashboard – returns to the welcome screen.
- Logout – safely ends the session.

4.5.4 Ransomware detection tool and interface

The ransomware detection tool is the core functionality of the web application, enabling users to upload and analyse malware feature data in real time. Built using the Flask framework, the interface allows users to evaluate .csv files containing extracted malware features via a machine learning model, which is a support vector machine optimised by particle swarm optimisation. After the upload, the

model processes the given input and determines the relevant ransomware family, making this tool an effective and interesting solution in the cybersecurity landscape.

This application helps bridge the gap between machine learning theories and real-world implementations in the area of cybersecurity. Unlike typical static reports, it demonstrates the integration of trained models into an in-depth, web-based application that enables users to interact with the system, submit inputs, and receive instantaneous, smart outputs.

The interface has been designed to be user-friendly and visually engaging. It includes:

- An information box describing the tool and supported input format.
- A file upload field, restricted to .csv files for safety and compatibility.
- A highlighted prediction result area, which dynamically displays the detected ransomware family.
- A detection history log, showing previously scanned files, results, and timestamps — useful for transparency and traceability.

Ransomware detection interface

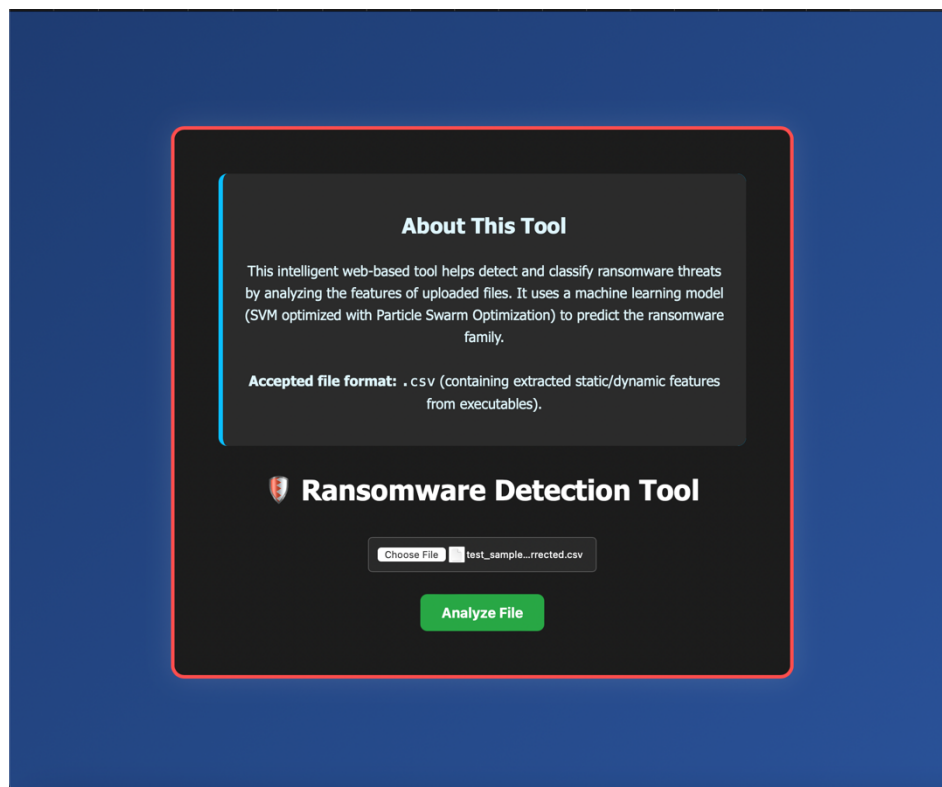


Figure 5.5

The backend handles the uploaded file by:

- Reading and validating its structure to ensure correct feature formatting.
- Applying the same StandardScaler used during model training.
- Passing the processed data to the trained SVM model.
- Displaying the result on-screen using a visually distinct prediction box.

predict_ransomware function

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
import joblib

# Load pre-trained model
model_path = os.path.join('model_training', 'svm_model_pso.pkl')
model = joblib.load(model_path)

# Reuse the same scaler used during training
scaler = StandardScaler()

def predict_ransomware(filepath):
    # 1. Load and validate structure
    df = pd.read_csv(filepath)

    if df.shape[1] != 55:
        raise ValueError(f"Expected 55 features, got {df.shape[1]}")

    # 2. Apply StandardScaler
    X_scaled = scaler.fit_transform(df)

    # 3. Predict with the loaded SVM model
    prediction = model.predict(X_scaled)

    # 4. Return the predicted label (string)
    return prediction[0]
```

Figure 5.6

This deployment is designed to bridge the gap between theoretical principles of machine learning and their application to cybersecurity practice. Unlike static reports, it demonstrates the integration of trained models into a complete, web-based system, allowing users to interact with the system, make inputs, and get immediate, intelligent responses.

4.6 Conclusion

Overall, the ransomware detection module consists of core system characteristics, enabling real-time classification of uploaded files into unique ransomware families through the utilization of an SVM model amplified using particle swarm optimization (PSO). The backend runs an asynchronous analysis of every file through inspection against its structure utilizing the same StandardScaler applied in the training process, prior to passing the data on to the trained model for prediction analysis—a procedure smoothly implemented in the Flask framework. The results are presented in an easily understandable format through the web interface, providing the user with instant and clear insight. The complete implementation documentation and code related to the above function can be viewed in Appendix A. The section describes the functional framework for the application and the entry into the next phase of the project—evaluation and testing—where the efficiency and effectiveness of the detection model is critiqued through practical measures and descriptive examples.

4.7 Results and testing

4.7.1 Purpose

This section critically evaluates the effectiveness and working competency of the deployed ransomware detection model using a suite of testing samples and known machine learning performance metrics. The ultimate goal here is to determine whether the trained model, after being integrated into the web application, is able to reliably make ransomware family predictions for unseen input in a reproducible manner. The assessment entails an analysis of the back-end model accuracy and the front-end usability, evidenced through live user interaction with the Flask-based interface.

4.7.2 Testing setup

The whole Flask web application was used in the assessment. The application identifies ransomware in real time when the users upload the .csv files. The files consist of features borrowed from malware, such as the process count, the thread count, memory usage data, and I/O operations. The features were extracted from the malware files with the help of static and dynamic analysis techniques.

The test samples were ensured to be actual and functional through taking and modifying them from the MalMem2022 dataset. It is an open dataset containing labelled ransomware samples belonging to varying groups. The files for the tests were modified to suit the distinctive format the SVM model, having 55 features trained on, would accept. It makes them function well together with the StandardScaler used for training.

All the test files were submitted via the simple interface provided with the Flask application. On submission, the system validated the data and processed it prior to the prediction based on the trained PSO-optimised Support Vector Machine (SVM) classifier. The prediction was displayed in the interface in the browser as a prediction box along with the detection history.

The functional testing objectives were carefully organised and mapped onto the project to verify if the project goal was successfully reached:

Testing objectives table

Test ID	Objective/Question	Expected Outcome	Test Result
T1	Can the system accept a .csv upload?	File is uploaded and processed without error	Pass
T2	Does the system correctly identify a ransomware type?	Prediction displays are a valid ransomware label	Pass to an extent
T3	Is invalid file input handled gracefully?	User receives an error message, no crash	Pass
T4	Is the detection history updated per upload?	New records appear with correct details	Pass

T5	Does the user interface remain usable and responsive?	UI elements work as expected after interaction	Pass
----	---	--	------

Table 3.1

Tested features

- 2 samples of each ransomware type were tested with different features in the .csv format. We tested on Ako, Conti, Maze, Pysa, Shade. With all these detections, the detection history was appended accordingly, as documented in previous sections.

Example different features (Ako)

pslist.nproc	pslist.nppid	pslist.avg_threads	pslist.nprocs64bit	pslist.avg_handlers	dlllist.ndlls	dlllist.avg_dlls_per_proc
42.01119087755930	18.001180950138500	11.194490630770300	1.0164214430059900	225.29782614356300	1599.9821917738300	41.498801705756600

Figure 6.1

pslist.nproc	pslist.nppid	pslist.avg_threads	pslist.nprocs64bit	pslist.avg_handlers	dlllist.ndlls	dlllist.avg_dlls_per_proc	ha
41.99501477060410	18.0137500831261	11.198936476524100	0.9987919083290660	225.3043582072410	1599.9921384775600	41.483786167349300	89

Figure 6.2

pslist.nproc	pslist.nppid	pslist.avg_threads	pslist.nprocs64bit	pslist.avg_handlers	dlllist.ndlls	dlllist.avg_dlls_per_proc
41.98824544700590	17.990585358499600	11.19979994504350	1.0049900826353200	225.30618044141100	1600.007935453840	41.48741202108640

Figure 6.3

Results (Ako)

These results show the consistency of the detection tool when detecting the Ako ransomware. It also confirmed other testing objectives, as the tool can successfully accept .csv files, the detection is incremented based on the time, and the user interface remained responsive.

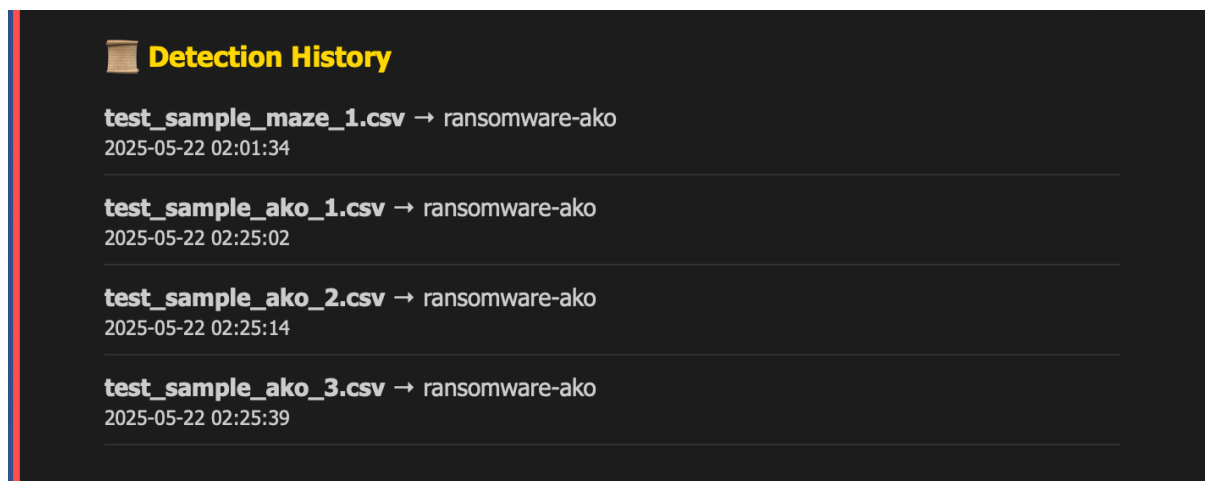


Figure 6.4

Summary

The ransomware detection mechanism has also been incorporated into a web interface developed on Flask so that it allows users to upload sample .csv test files for classification. The backend employs a Support Vector Machine-optimized classifier and uses Particle Swarm Optimization (PSO) for facilitating multiclass classification between existing ransomware types—namely, Ako, Maze, Conti, Shade, and Pysa. Although the detection rate has been proven on a set of Ako instances—because they dominated the dataset—differentiating between highly related ransomware variants like Conti and Maze has proven a challenging task. The number of misclassifications of those variants may be due to the fact that they comprise overlapping features and possibly class imbalances in the available dataset. Additionally, the tool has proven highly efficient in processing legitimate inputs and generating expected results in a timely manner as well as eliminating invalid or incomplete test inputs efficiently. All related code and feature engineering practices are explained in Appendix A. The next section offers a thorough analysis of the results as well as a performance analysis.

Chapter 5 - Discussion and analysis

5.1 Discussion

The ransomware detector suggested here could effectively utilise a PSO-optimized Support Vector Machine (SVM) classifier through a web application developed on Flask. By testing it on various samples taken from the MalMem2022 dataset, the system could effectively demonstrate the capacity for ransomware detection and categorization according to static and dynamic features obtained from malware samples.

Also predicted by the model were ransomware-ako at very confident levels, and although it struggled somewhat in distinguishing between the very related maze and conti lineages from each other, the predictions aligned well with the distribution of training data and indicated the model has learnt significant patterns. This accords well with the applicability of machine learning in real-life threat detection systems when features have carefully been crafted and optimised through bio-inspired optimisation.

The incorporation into a real-time web platform also tested the usability of the model, where users were able to upload samples and receive instant feedback. The interface also logged predictions along with time stamps, which provided an open audit trail and allowed post-detection analysis.

5.2 Limitations

Class Imbalance and Model Bias: The most significant of the deficits found was the model's bias toward ransomware-ako prediction. The bias can probably be attributed to class imbalance in the training set, and some classes such as ako may be over-represented compared to the other ones, like maze or conti.

Input Format Restrictions: The system assumes preprocessed .csv files containing exactly 55 features—having a similar format as the training data. This makes the system less adaptable and inconvenient for other sources of data or future malware samples having varied formats.

No Native Executable Analysis: The tool accepts .exe executables as input not due to feasibility or safety considerations but rather from working on pre-extracted features. While this ensures safety from

security threats, it also bars actual real-world application where analysts generally handle unparsed malware.

Misclassification of Similar Families: Certain ransomware samples (e.g., maze) were misclassified as ako when they were tested. It may be an indication of common feature patterns between the families and could call for improving feature engineering or the use of deep-learning based discriminators as future research.

Faulty Inputs and NaN: The model fails when the imported CSV has missing values (NaN), without fallback preprocessing. Robustness is compromised even when already manually checked error detection on invalid test files.

5.3 Summary

As a whole, the deployment of a real-time ransomware detector through ML and Flask proved very successful. The model detected and categorised instances of ransomware well through a web interface. The results were largely accurate, and the predictions were also able to identify instances of the train families well, particularly ransomware-ako.

The tool proves to be of actual utility through the functionality of processing and analysing structured input data in real time. Detection of less common ransomware families did reveal a requirement for better balanced datasets and adjustment of the models. The solution, despite imperfections, remains a good starting point for deployment to production in lightweight malware analysis processes.

Chapter 6 - Conclusions and future work

6.1 Conclusion

The main aim of this research effort was to create and implement a ransomware detection system based on machine-learning methods capable of detecting specific ransomware families in real time via a web interface. The system consists of a PSO-optimized SVM and a Flask frontend allowing for the upload of feature files and the provision of instantaneous detection reports.

The integration of the trained model within a web framework has also proved to be effective, helping the tool make accurate predictions on a number of test instances. Analysis on the well-organized .csv instances obtained from the MalMem2022 dataset showed the effectiveness of the tool in detecting ransomware attacks, especially ransomware-ako, resulting in accurate outcomes.

Though some specific limitations were identified—specifically related to the discrimination between highly related ransomware groups like Maze and Conti—the system performed its core operations in a stable and regular way. Additionally, the web application provided an easy-to-use interface and a clear presentation of detection history and thus enhanced usability and transparency.

In essence, this work provides a bridge between theoretical machine learning models and their usability through easy-to-use and accessible software tools. It also demonstrates the viability of integrating powerful classification algorithms and efficient web applications in building useful and scalable solutions for security.

6.2 Future Work

Though the present system runs well as per criteria set out beforehand, considerable room for improvement in the future exists.

Augmented Dataset and Balanced Class Representation: The salient problem noted here was the imbalanced class representation of some ransomware groups, primarily ransomware-ako. Future research activities must aim at achieving a fair dataset that preserves equal and sufficient representation in the diverse ransomware categories.

Analysis of Executable Files: The current tool is limited to examination of .csv files containing features that have already been extracted. A critical improvement would be found in the addition or implementation of a preprocessing engine that would be able to extract features from .exe files in a secure sandbox environment directly. This facility would greatly enhance the tool's utility in real-world contexts in dynamic security environments.

Advanced Model Architectures: Incorporating stronger classification approaches, such as ensemble methods, XGBoost, or deep learning models (e.g., Convolutional Neural Networks or Recurrent Neural Networks), could increase accuracy—particularly in the case of unclear or rare ransomware families.

Error Management and User Response: Providing adequate feedback on unsuccessful upload operations—e.g., due to missing values or inaccurate formatting—would improve application functionality as a whole. Pre-upload checks and offering suggestions for correction would lend themselves toward achieving a higher level of fault tolerance in the system.

Dynamic Re-training and Model Updating: In real-world scenarios, ransomware is continuously evolving. Follow-up research can incorporate mechanisms for continuous re-training based on newly obtained datasets or crowdsourced threat intelligence, thus allowing the tool to remain effective against emerging challenges.

Increased web integration: The current web interface functions well, but future extensions might include:

Real-time API integration for automatic scanning

Role-based access control combined with administrative interfaces

Threat-level visualisations and confidence scores for predictions Hybrid Detection Capabilities: Finally, combining this machine learning-based static analysis framework with dynamic behavioural analysis—e.g., using information gathered from sandboxed execution—can improve the detection of zero-day or polymorphic ransomware variants. Finally, the established system in this project offers a strong foundation for ransomware automation and categorisation of ransomware variants. As future development focuses on improving data diversity levels, adapting model architecture for optimisation,

and improving real-time data processing capacity, this solution can evolve as a financially sustainable cybersecurity tool.

Chapter 7

7.1 Reflection

Starting this capstone project represented a seminal moment in my academic path. It provided a unique challenge for integrating the skills learnt through my computer science curriculum in a single and holistic effort. By addressing a particular problem in the area of cybersecurity and creating a single system for ransomware detection through machine learning strategies, this capstone has refined both my technical expertise and my analytical skills.

The launch of the project was marked by a clear goal: the creation of a smart system able to detect and classify ransomware attacks through the application of machine learning algorithms. It started off from the requirement of clearly defining the scope of the project. The huge area of malware detection required a focus on a more feasible and meaningful goal, eventually transforming into the main goal of classifying ransomware strains based on both static and dynamic characteristics.

One of the most rewarding aspects of the project was gaining hands-on experience in working with machine learning models. At the beginning, a number of challenges, mainly concerning dataset preparation and class distribution imbalances in the MalMem2022 dataset, presented themselves. Understanding the impact of imbalances on classification performance brought home the value of data curation and preprocessing. The implementation of Particle Swarm Optimisation (PSO) on the optimisation of my Support Vector Machine (SVM) model posed a significant technical challenge but also proved to be one of the most insightful experiences. This task brought home the possible application of metaheuristics for optimising model performance in real-world settings.

Another determining factor was the web interface. Having had minimal background experience in web development when I started this project, I was able to build a full-fledged application with Flask that allowed users to upload ransomware-related files and get predictions in real-time. This phase pushed me to move out of my set comfort zone and required me to learn quickly about integrating back-end functionality with front-end user experience. The final product was not merely functional but also exhibited user-friendliness—two qualities of which I am proud to have achieved within a limited time span.

Throughout the project, I gained a much deeper appreciation for system design and testing. I realised that building a model represents only one facet of the challenge; the successful incorporation of that model into a working application, the verification of inputs, the elegant handling of errors, and the delivery of a smooth user experience are equally important. I was also forced to consider approaches to making predictions both explainable and traceable, and in doing so, building user trust in the system.

The time-management concept came as a hard-earned lesson. Balancing this project alongside other university assignments and academic commitments required a lot of discipline and planning. While I mostly managed to stick to the schedule, moments of delay in training or unexpected hurdles in achieving Flask forced me to re-evaluate priorities in my task list. Nevertheless, the challenges taught me the value of perseverance and the importance of making incremental progress.

Looking back at this experience, it is clear that the project has improved my understanding of applied machine learning, cybersecurity, web development, and project management. In addition, I have more confidence in presenting technical presentations in both oral and written forms. If I were to work on the project again, I would improve the detection capabilities of underrepresented ransomware families and expand the system to allow real-time executable scanning. At the core, this initiative has bettered my technical skills and prepared me for tackling complex challenges faced in real-life settings. I gained the necessary skills and self-assurance required to advance on a professional path related to cybersecurity or artificial intelligence and make meaningful contributions. The experience, rigor, and lessons learned through the course of this project will certainly create a solid foundation for my future academic or professional endeavours.

Chapter 8

8.1 Legal and regulatory compliance

8.1.1 Social Impact

This project complies with applicable legal frameworks and regulatory standards regarding data use, intellectual property rights, and digital rights.

- **Data Governance:** The data used in this research (MalMem2022) was gathered from publicly available research for academic use. No personal or sensitive user data was processed.
- **Intellectual Property:** All third-party tools and datasets were properly cited and used within specified usage guidelines (i.e., open-source licensing agreements).
- **Liability and Accountability:** The current project is a research tool, and any deployment in practical situations requires the creation of formal disclaimers, addition of security measures, and legal review to duly assign liability.

8.1.2 Ethical Framework

The framework rests on fundamental ethical standards of transparency, fairness, and accountability.

- **Consent and Privacy:** As no private user data was involved, there were no ethical concerns around consent. However, any future extension involving personal data would require adherence to data protection laws such as GDPR.
- **Transparency:** The report and the supporting material offer thorough documentation of the system's workings, including data preprocessing, the adopted model, and the classification method used.
- **Code of Conduct:** Throughout development, we maintained adherence to established ethical machine learning guidelines, which included fair practices in model evaluation and avoiding making misleading claims about accuracy or detection capabilities.

8.1.3 Risk Management

Potential project risks were identified and addressed:

- Risk of Bias: The dataset was examined for balance across ransomware families. However, minor skew (e.g., more samples for ransomware-ako) could affect classification accuracy. This was documented and partially mitigated through stratified sampling and validation.
- Model Misclassification: The tool has a tendency to misclassify certain ransomware categories unintentionally. This is a known limitation of machine learning approaches and should be clearly explained in any deployment.
- Security Issues: While the Flask web application has been written using secure programming practices, it has not been properly hardened for deployment in a malicious environment. The application of proper file sanitisation procedures and constrained execution contexts would be essential for use in production.
- Mitigation Strategies: Future versions may include: Model confidence scores to flag uncertain predictions. Tracking and alerting on anomalous upload behaviour. Progressive retraining with new data instances.

Gitlab link: https://gitlab.act.reading.ac.uk/fh002356/ransomware-detection/-/tree/student-submission?ref_type=heads

Bibliography

- AHN, G.-J., YI, J.H. and CHO, H. (2018) *(PDF) dexmonitor: Dynamically analyzing and monitoring obfuscated Android Applications, DexMonitor: Dynamically Analyzing and Monitoring Obfuscated Android Applications*. Available at: https://www.researchgate.net/publication/329006904_DexMonitor_Dynamically_Analyzing_and_Monitoring_Obfuscated_Android_Applications (Accessed: 08 May 2025).
- Aldhaffer, N. (2024) *Android malware detection using support vector regression for dynamic feature analysis, MDPI*. Available at: <https://www.mdpi.com/2078-2489/15/10/658> (Accessed: 23 May 2025).
- Anderson, B. et al. (2011) *Graph-based malware detection using dynamic analysis - Journal of Computer Virology and hacking techniques, SpringerLink*. Available at: <https://link.springer.com/article/10.1007/s11416-011-0152-x> (Accessed: 10 May 2025).
- Arief, B., Pont, J. and Hernandez-Castro, J. (2020) *(PDF) why current statistical approaches to ransomware detection fail, Why Current Statistical Approaches to Ransomware Detection Fail*. Available at: https://www.researchgate.net/publication/347262897_Why_Current_Statistical_Approaches_to_Ransomware_Detection_Fail (Accessed: 23 May 2025).
- Aslan, O. and Samet, R. (2019) *A comprehensive review on malware detection approaches | IEEE Journals & Magazine | IEEE Xplore*. Available at: <https://ieeexplore.ieee.org/document/8949524/> (Accessed: 28 October 2024).
- Author links open overlay panelColin Tankard et al. (2011) *Advanced persistent threats and how to monitor and deter them, Network Security*. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S1353485811700861> (Accessed: 04 March 2025).
- Baker, K. (2022) *What is a polymorphic virus? examples & more, CrowdStrike*. Available at: <https://www.crowdstrike.com/en-us/cybersecurity-101/malware/polymorphic-virus/> (Accessed: 23 May 2025).
- Biau, G. and Scornet, E. (2016) *A Random Forest Guided Tour - Test, SpringerLink*. Available at: <https://link.springer.com/article/10.1007/s11749-016-0481-7> (Accessed: 23 May 2025).
- Brook, C. (2024) *50 examples of ransomware attacks and their impacts, Enterprise IP and DLP Software*. Available at: <https://www.digitalguardian.com/blog/50-examples-ransomware-attacks-and-their-impacts> (Accessed: 23 May 2025).
- Brosolo, M. et al. (2025) *Security through the eyes of AI: How visualization is shaping malware detection, arXiv.org*. Available at: <https://arxiv.org/abs/2505.07574> (Accessed: 23 May 2025).
- Dhurjati, D., Bendre, M. and Sekar, R. (2001) *A fast automaton-based method for detecting anomalous program behaviors | IEEE conference publication | IEEE Xplore*. Available at: <https://ieeexplore.ieee.org/document/924295/> (Accessed: 06 March 2025).

- Eisa, M., Witherspoon, R. and Yardley, Q. (2025) *Semantic Entanglement-Based Ransomware Detection via Probabilistic Latent Encryption Mapping, Semantic entanglement-based ransomware detection via probabilistic latent encryption mapping*. Available at: <https://arxiv.org/html/2502.02730v1> (Accessed: 05 March 2025).
- El-Taweel, G.S., Mead, M.A. and Salem, H.S.E.-T.S. (2024) *Particle swarm optimization-based hyperparameters tuning of machine learning models for big COVID-19 data analysis, Particle Swarm Optimization-Based Hyperparameters Tuning of Machine Learning Models for Big COVID-19 Data Analysis*. Available at: <https://www.scirp.org/journal/paperinformation?paperid=132026> (Accessed: 23 May 2025).
- Firdaus, A. et al. (2017) *Bio-inspired computational paradigm for Feature Investigation and Malware Detection: Interactive Analytics - multimedia tools and applications, SpringerLink*. Available at: <https://link.springer.com/article/10.1007/s11042-017-4586-0> (Accessed: 11 May 2025).
- Gandotra, E. and Gupta, D. (2021) *An efficient approach for phishing detection using machine learning, SpringerLink*. Available at: https://link.springer.com/chapter/10.1007/978-981-15-8711-5_12 (Accessed: 20 February 2025).
- Gelinas, R.R. (2010) *Cyberdeterrence and the problem of attribution, DG*. Available at: <https://repository.digital.georgetown.edu/handle/10822/553494> (Accessed: 05 March 2025).
- Gomez-hernandez, J.A. et al. (2017) *R-locker: Thwarting ransomware action through a honeyfile-based approach, Computers & Security*. Available at: <https://www.sciencedirect.com/science/article/pii/S0167404817302560> (Accessed: 07 February 2025).
- Guo, Y. (2022) *A review of machine learning-based zero-day attack detection: Challenges and future directions, Computer Communications*. Available at: https://www.sciencedirect.com/science/article/pii/S0140366422004248?casa_token=N1D_tMD_huoAAAAA%3Awsea8g2_CSp6VZ-oKfdtyRHlwqjGGHkHfTlctAGfg2NLXrdMEG13FO1j3FCB8os1hmA901Uj (Accessed: 04 March 2025).
- Herrera-Silva, J.A. and Hernández-Álvarez, M. (2023) *Dynamic feature dataset for ransomware detection using machine learning algorithms, MDPI*. Available at: <https://www.mdpi.com/1424-8220/23/3/1053> (Accessed: 23 May 2025).
- Huang, Z. and Zhang, Y. (2024) *Unveiling Hidden Patterns: A Computational Analysis of Less Commonly Labeled Ransomware Families*. Available at: <https://www.authorea.com/users/817469/articles/1217790-unveiling-hidden-patterns-a-computational-analysis-of-less-commonly-labeled-ransomware-families> (Accessed: 19 February 2025).
- Humayun, M., Alsayat, A. and Ponnusamy, V. (2020) *Internet of things and Ransomware: Evolution, mitigation and prevention, Egyptian Informatics Journal*. Available at: <https://www.sciencedirect.com/science/article/pii/S1110866520301304> (Accessed: 05 May 2025).

- Hunt, G. (2016) *Locky ransomware spreads via fake invoice spam at an alarming rate.*, TitanHQ. Available at: <https://www.titanhq.com/blog/locky-ransomware-spreads-via-fake-invoice-spam-at-an-alarming-rate/> (Accessed: 23 May 2025).
- Ikuse, T. et al. (2016) *Malware detection with deep neural network using ...*, *Malware Detection with Deep Neural Network Using Process Behavior*. Available at: https://www.researchgate.net/publication/306927503_Malware_Detection_with_Deep_Neural_Network_Using_Process_Behavior (Accessed: 10 May 2025).
- JOSEPH, L. (2011) (PDF) *a hybrid algorithm to detect malware and eliminate zero day attacks by / LWOMWA Joseph and John Ngubiri - academia.edu, A HYBRID ALGORITHM TO DETECT MALWARE AND ELIMINATE ZERO DAY ATTACKS*. Available at: https://www.academia.edu/43345112/A_HYBRID_ALGORITHM_TO_DETECT_MALWARE_AND_ELIMINATE_ZERO_DAY_ATTACKS_by (Accessed: 07 May 2025).
- Karim, N. (2024) *Comprehensive analysis of ransomware evolution and countermeasures in the era of Digital Transformation, International Journal of Advanced Cybersecurity Systems, Technologies, and Applications*. Available at: <http://theaffine.com/index.php/IJACSTA/article/view/3> (Accessed: 05 May 2025).
- Kaspersky (2018) *What is heuristic analysis?, What is Heuristic Analysis?* Available at: <https://www.kaspersky.com/resource-center/definitions/heuristic-analysis#> (Accessed: 07 May 2025).
- Kharraz, A. et al. (1970) *Cutting the gordian knot: A look under the hood of ransomware attacks*, SpringerLink. Available at: https://link.springer.com/chapter/10.1007/978-3-319-20550-2_1 (Accessed: 18 February 2025).
- Khatoun, R., Mustafa, A. and Chbib, F. (2024) *Leveraging machine learning-based PDF malware detection in Snort | IEEE conference publication | IEEE Xplore, Leveraging Machine Learning-Based PDF Malware Detection in Snort*. Available at: <https://ieeexplore.ieee.org/document/10796480/> (Accessed: 01 March 2025).
- Kothamali, P.R. and Banik, S. (2022) *Limitations of signature-based threat detection*, *Revista de Inteligencia Artificial en Medicina*. Available at: <http://redcrevistas.com/index.php/Revista/article/view/131> (Accessed: 14 January 2025).
- Labiche, Y. and Massicotte, F. (2012) *On the verification and validation of signature-based, Network Intrusion Detection Systems | IEEE conference publication | IEEE Xplore, On the Verification and Validation of Signature-Based, Network Intrusion Detection Systems*. Available at: <https://ieeexplore.ieee.org/document/6405405> (Accessed: 01 March 2025).
- LaRocque, A. et al. (2024) *Effective Ransomware Detection Using Autonomous Pattern-based Signature Extraction*. Available at: <https://www.authorea.com/doi/full/10.22541/au.173016272.26231350> (Accessed: 05 May 2025).
- Li, T., Wang, D. and Ye, Y. (2007) (PDF) *IMDS: Intelligent malware detection system, IMDS: intelligent malware detection system*. Available at: https://www.researchgate.net/publication/221654267_IMDS_Intelligent_malware_detection_system (Accessed: 08 May 2025).

- MARY, C. et al. (2024) *A review on cognitive based ransomware ...*, *A REVIEW ON COGNITIVE BASED RANSOMWARE DETECTION USING MACHINE LEARNING AND DEEP LEARNING TECHNIQUES*. Available at: <http://www.jatit.org/volumes/Vol102No10/30Vol102No10.pdf> (Accessed: 20 February 2025).
- Mehdi, S., Hashemi, H. and Bazrafshan, Z. (2013) (PDF) *A survey on Heuristic Malware Detection Techniques*, *A survey on Heuristic Malware Detection Techniques*. Available at: https://www.researchgate.net/publication/260729684_A_survey_on_heuristic_malware_detection_techniques (Accessed: 05 March 2025).
- MOHSEN, F. et al. (2024) *Hybrid android malware detection: A review of heuristic-based approach* | *IEEE Journals & Magazine | IEEE Xplore*, *Hybrid Android Malware Detection: A Review of Heuristic-Based Approach*. Available at: <https://ieeexplore.ieee.org/document/10473005/> (Accessed: 07 May 2025).
- Narayan, S. and Pandiyan, N. (2023) *A survey on Deep Learning models embed bio-inspired algorithms in Cardiac Disease Classification*, *A Survey on Deep Learning Models Embed Bio-Inspired Algorithms in Cardiac Disease Classification*. Available at: <https://openbiomedicalengineeringjournal.com/VOLUME/17/ELOCATOR/e187412072212261/FULLTEXT/> (Accessed: 11 May 2025).
- Nath, H., Menon, R. and Radhakrishnan, K. (2019) *A survey of Zero-day malware attacks and its detection methodology* | *IEEE conference publication | IEEE Xplore*, *A survey of zero-day malware attacks and its detection methodology*. Available at: <https://ieeexplore.ieee.org/document/8929620/> (Accessed: 05 March 2025).
- Nguyen, G. et al. (2018) *A heuristics approach to mine behavioural data logs in Mobile Malware Detection System*, *Data & Knowledge Engineering*. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0169023X17303063> (Accessed: 08 May 2025).
- Patil, M. and Mohurle, S. (2017) *A brief study of wannacry threat: Ransomware attack 2017*. Available at: <https://sbgsmmedia.in/2018/05/10/2261f190e292ad93d6887198d7050dec.pdf> (Accessed: 18 February 2025).
- Pont, J. and Hernandez-Castro, J. (2020) (PDF) *why current statistical approaches to ransomware detection fail*, *Why Current Statistical Approaches to Ransomware Detection Fail*. Available at: https://www.researchgate.net/publication/347262897_Why_Current_Statistical_Approaches_to_Ransomware_Detection_Fail (Accessed: 23 May 2025).
- Razak, M.F.A. et al. (2017) *Bio-inspired for features optimization and malware detection - arabian journal for science and engineering*, *SpringerLink*. Available at: <https://link.springer.com/article/10.1007/s13369-017-2951-y> (Accessed: 11 May 2025).
- Reddy, K.T. (2024) *Unveiling the strengths and limitations of signature based phishing detection*, *Insights2Techinfo*. Available at: <https://insights2techinfo.com/unveiling-the-strengths-and-limitations-of-signature-based-phishing-detection/> (Accessed: 27 October 2024).

- Richardson, R. and North, M.M. (2017) *Ransomware: Evolution, mitigation and prevention, DigitalCommons@Kennesaw State University*. Available at: <https://digitalcommons.kennesaw.edu/facpubs/4276/> (Accessed: 14 January 2025).
- Rubens, P. (2021) *Open source software's role in breach prevention and detection, eSecurity Planet*. Available at: <https://www.esecurityplanet.com/applications/open-source-softwares-role-in-breach-prevention-and-detection/> (Accessed: 01 March 2025).
- Santos, I. et al. (2011) *Opcode sequences as representation of executables for data-mining-based unknown malware detection, Information Sciences*. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0020025511004336> (Accessed: 08 May 2025).
- SentinelOne (2021) *What is a malware signature and how does it work?, SentinelOne*. Available at: <https://www.sentinelone.com/blog/what-is-a-malware-file-signature-and-how-does-it-work/> (Accessed: 23 May 2025).
- Sgandurra, D. et al. (2016) *Automated Dynamic Analysis of ransomware: Benefits, limitations and use for detection, arXiv.org*. Available at: <https://arxiv.org/abs/1609.03020> (Accessed: 09 May 2025).
- Ucci, D., Aniello, L. and Baldoni, R. (2018) *Survey of machine learning techniques for malware analysis, Computers & Security*. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0167404818303808> (Accessed: 09 May 2025).
- Ullah, F. et al. (2017) *Data Exfiltration: A review of external attack vectors and countermeasures, Journal of Network and Computer Applications*. Available at: <https://www.sciencedirect.com/science/article/pii/S1084804517303569> (Accessed: 21 February 2025).
- Verma, A. and Ranga, V. (2019) *Machine learning based intrusion detection systems for IOT applications - wireless personal communications, SpringerLink*. Available at: <https://link.springer.com/article/10.1007/S11277-019-06986-8> (Accessed: 10 May 2025).
- Yaseen, A. (2020) *UNCOVERING EVIDENCE OF ATTACKER BEHAVIOR ON THE NETWORK*. Available at: https://www.researchgate.net/profile/Asad-Yaseen-2/publication/378594218_UNCOVERING_EVIDENCE_OF_ATTACKER_BEHAVIOR_ON_THE_NETWORK/links/65e1280aadf2362b635f771d/UNCOVERING-EVIDENCE-OF-ATTACKER-BEHAVIOR-ON-THE-NETWORK.pdf (Accessed: 2025).
- Yuryna Connolly, L. et al. (2020) *An empirical study of ransomware attacks on organizations: An assessment of severity and salient factors affecting vulnerability, OUP Academic*. Available at: <https://academic.oup.com/cybersecurity/article/6/1/tyaa023/6047253> (Accessed: 07 November 2024).

Appendix A (Machine learning code + graphs)

```
import pandas as pd

# Load dataset
df_full = pd.read_csv("MalMem2022.csv")

# Clean the 'Category' column to avoid casing/spacing issues
df_full['Category'] = df_full['Category'].str.strip().str.lower()

# Filter ransomware samples (those that start with 'ransomware')
ransom_df = df_full[df_full['Category'].str.startswith('ransomware')].copy()

# Check output
print("✅ Total ransomware samples:", ransom_df.shape[0])
print("👁️ Ransomware families:", ransom_df['Category'].unique())
ransom_df.head()

# === Preprocessing code block ===

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# --- Encode ransomware families ---
label_encoder = LabelEncoder()
ransom_df['family_label'] = label_encoder.fit_transform(ransom_df['Category'])

# Optional: see the mapping
label_map = dict(zip(label_encoder.classes_, label_encoder.transform(label_encoder.classes_)))
print("📄 Label Mapping:", label_map)

# --- Drop unused columns ---
ransom_df = ransom_df.drop(columns=['Class', 'Category', 'Filename'])

# --- Define features and labels ---
X = ransom_df.drop(columns=['family_label'])
y = ransom_df['family_label']

# --- Train/test split ---
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)

# --- Scaling ---
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Appendix A.1 – Data Preprocessing Pipeline for Ransomware Classification

```

# === Random Forest: Ransomware Family Classification ===

import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

# Load dataset (update path if needed)
df = pd.read_csv("MalMem2022.csv")

# Filter only ransomware samples based on the correct column
ransomware_df = df[df['Class'].str.lower() == 'malware']
ransomware_df = ransomware_df[ransomware_df['Category'].str.lower().str.star

# Extract features and target
X = ransomware_df.drop(columns=['Class', 'Category', 'Filename']) # Feature:
y = ransomware_df['Category'].str.lower() # Ransomware families (target)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)

# Normalize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Random Forest
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_scaled, y_train)

# Predict and evaluate
y_pred_rf = rf_model.predict(X_test_scaled)

print("=== Confusion Matrix (Random Forest) ===")
print(confusion_matrix(y_test, y_pred_rf))

```

```

print("\n=== Classification Report (Random Forest) ===")
print(classification_report(y_test, y_pred_rf))

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

# Compute confusion matrix
cm = confusion_matrix(y_test, y_pred_rf)

# Plot heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=rf_model.classes_,
            yticklabels=rf_model.classes_)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Random Forest (Ransomware Families)')
plt.show()

import numpy as np

# Get feature importances and top indices
importances = rf_model.feature_importances_
indices = np.argsort(importances)[-10:] # Top 10

# Plot
plt.figure(figsize=(10, 6))
plt.barh(range(len(indices)), importances[indices], align='center')
plt.yticks(range(len(indices)), [X.columns[i] for i in indices])
plt.xlabel('Feature Importance Score')
plt.title('Top 10 Important Features - Random Forest')
plt.tight_layout()
plt.show()

```

Appendix A.2 - Random Forest Classifier Implementation


```
[ ] # === KNN classifier ===

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix

# === Initialize and train the KNN model ===
knn_model = KNeighborsClassifier(n_neighbors=1) # You can tune this later
knn_model.fit(X_train, y_train)

# === Predict on the test set ===
y_pred_knn = knn_model.predict(X_test)

# === Evaluate the model ===
print("=== Confusion Matrix (KNN) ===")
print(confusion_matrix(y_test, y_pred_knn))

print("\n=== Classification Report (KNN) ===")
print(classification_report(y_test, y_pred_knn))

import matplotlib.pyplot as plt

k_range = range(1, 11)
cv_scores = []

for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X, y, cv=5, scoring='accuracy')
    cv_scores.append(scores.mean())

plt.figure(figsize=(8, 5))
plt.plot(k_range, cv_scores, marker='o')
plt.xlabel("k (Number of Neighbors)")
plt.ylabel("Cross-Validated Accuracy")
plt.title("KNN Accuracy for Different k Values")
plt.grid(True)
plt.show()

# === Confusion Matrix Heatmap ===

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred_knn)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=knn_model.classes_,
            yticklabels=knn_model.classes_)
plt.title("KNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

Appendix A.3 - KNN classifier, heatmap and confusion matrix

```

""" # Cross validation to tune k - finding out the best value for k

from sklearn.model_selection import cross_val_score

print("Tuning k (1 to 10)...")
for k in range(1, 11):
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X, y, cv=5, scoring='accuracy')
    print(f"k = {k} → Mean Accuracy: {scores.mean():.4f}")

"""

import matplotlib.pyplot as plt

k_range = range(1, 11)
cv_scores = []

for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X, y, cv=5, scoring='accuracy')
    cv_scores.append(scores.mean())

plt.figure(figsize=(8, 5))
plt.plot(k_range, cv_scores, marker='o')
plt.xlabel("k (Number of Neighbors)")
plt.ylabel("Cross-Validated Accuracy")
plt.title("KNN Accuracy for Different k Values")
plt.grid(True)
plt.show()

```

Appendix A.3.1 – Cross-validation of 'k', finding best k

```

# === Confusion Matrix Heatmap ===

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred_knn)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=knn_model.classes_,
            yticklabels=knn_model.classes_)
plt.title("KNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()

```

Appendix A.3.2 – Heatmap for KNN

```

# === Baseline SVM classifier ===

from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix

# Train the SVM model (multiclass classification)
svm_model = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm_model.fit(X_train, y_train)

# Predict
y_pred_svm = svm_model.predict(X_test)

# Evaluate
print("=== Confusion Matrix (SVM) ===")
print(confusion_matrix(y_test, y_pred_svm))

print("\n=== Classification Report (SVM) ===")
print(classification_report(y_test, y_pred_svm))

```

Appendix A.4 – SVM classifier (Baseline)

```

# === SVM with PSO ===

from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score
import numpy as np
import pyswarms as ps

# Define PSO objective function (minimizing 1 - cross-validated accuracy)
def objective_function(params):
    C = params[:, 0]
    gamma = params[:, 1]
    scores = []

    for i in range(len(C)):
        model = SVC(kernel='rbf', C=C[i], gamma=gamma[i], random_state=42)
        score = cross_val_score(model, X_train_scaled, y_train, cv=3, scoring='accuracy')
        scores.append(1 - score) # PSO minimizes, so we want 1 - accuracy

    return np.array(scores)

# Define bounds: [C, gamma]
bounds = ([0.1, 0.0001], [100, 1]) # C from 0.1 to 100, gamma from 0.0001 to 1

# Configure and run PSO
optimizer = ps.single.GlobalBestPSO(
    n_particles=10,
    dimensions=2,
    options={'c1': 0.5, 'c2': 0.3, 'w': 0.9},
    bounds=bounds
)

best_cost, best_pos = optimizer.optimize(objective_function, iters=10)

# Train final SVM model with best C and gamma
best_C, best_gamma = best_pos
svm_pso = SVC(kernel='rbf', C=best_C, gamma=best_gamma, random_state=42)
svm_pso.fit(X_train_scaled, y_train)

```

Appendix A.4.1 – Optimisation with bio-heuristic algorithms (Particle Swarm Opt.)

```

# Predict and evaluate
y_pred_pso = svm_pso.predict(X_test_scaled)

from sklearn.metrics import classification_report, confusion_matrix
print("=== Confusion Matrix (SVM + PSO) ===")
print(confusion_matrix(y_test, y_pred_pso))

print("\n=== Classification Report (SVM + PSO) ===")
print(classification_report(y_test, y_pred_pso))

# === Classification performance bar ===

import matplotlib.pyplot as plt
import numpy as np

classes = ['ako', 'conti', 'maze', 'pysa', 'shade']
f1_before = [0.32, 0.02, 0.00, 0.13, 0.39]
f1_after = [0.48, 0.49, 0.50, 0.38, 0.58]

x = np.arange(len(classes))
width = 0.35

plt.figure(figsize=(10, 6))
plt.bar(x - width/2, f1_before, width, label='SVM')
plt.bar(x + width/2, f1_after, width, label='SVM + PSO')
plt.ylabel('F1-Score')
plt.xlabel('Ransomware Family')
plt.title('F1-Score per Ransomware Class (Before vs After PSO)')
plt.xticks(x, classes)
plt.legend()
plt.tight_layout()
plt.show()

```

Appendix A.4.2 – Classification bar SVM + PSO